

8 Automated EDA Tools.....	443
2.16) Data Visualisation.....	444
Most Important Plots in Data Science.....	444
How are QQ Plots Created?.....	451
8 Elegant Alternatives to Traditional Plots.....	456
Interactive Controls.....	464
Mosaic Plots.....	465
Enrich Matplotlib Plots with Inset Axis and Annotations.....	466
Professionalize Matplotlib Plots.....	469
Sankey Diagrams.....	471
Ridgeline Plots.....	474
Sparkline Plots.....	477
2.17) SQL.....	479
Grouping Sets, Rollup and Cube in SQL.....	479
Semi, Anti, and Natural Joins.....	485
Use SQL "NOT IN" With Caution.....	491
2.18) Python OOP.....	496
Getters and Setters.....	496
Descriptors in Python.....	499
20 Most Common Magic Methods.....	509
Memory Efficient Class Objects using Slots.....	510
Why Don't We Invoke model.forward() in PyTorch?.....	515
True OOP Encapsulation is Missing From Python.....	520
A Common Misconception About __init__().....	524
Function Overloading in Python.....	527

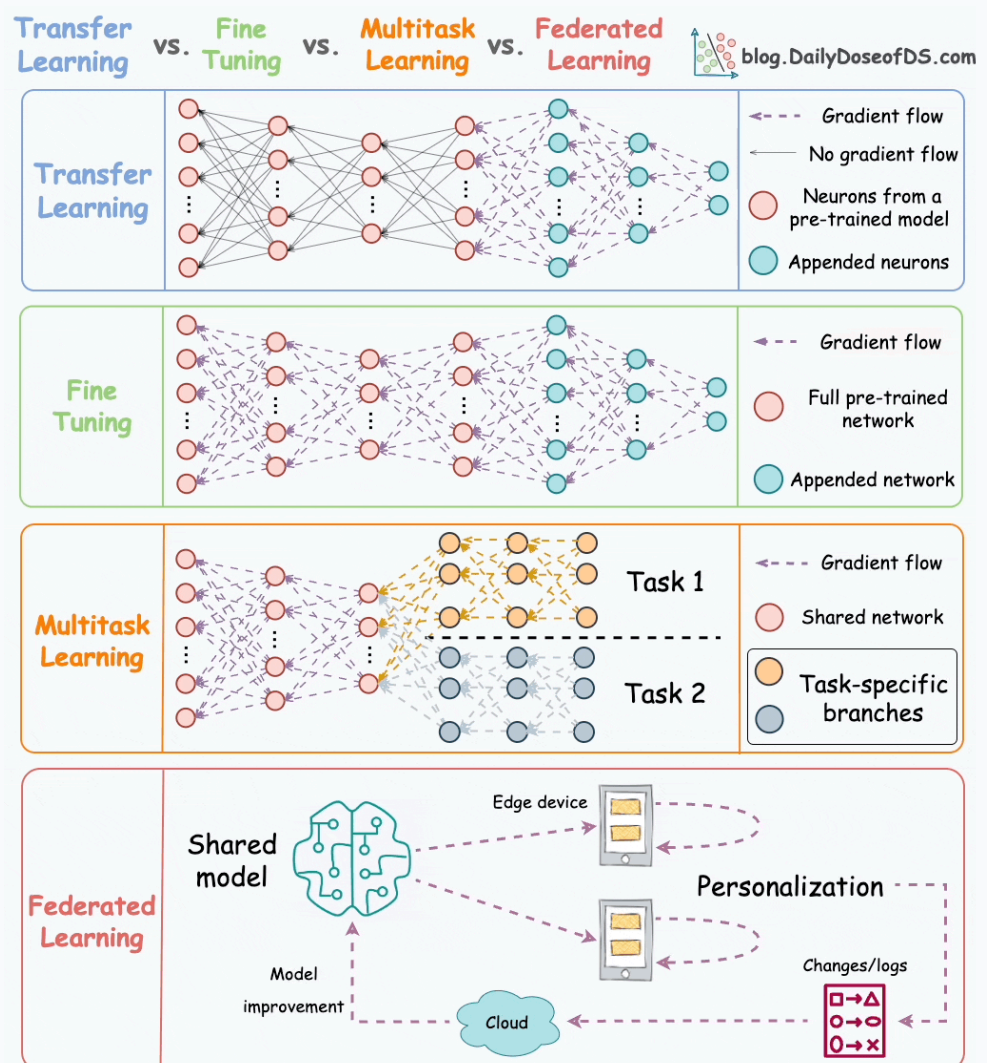
Deep learning

Learning Paradigms

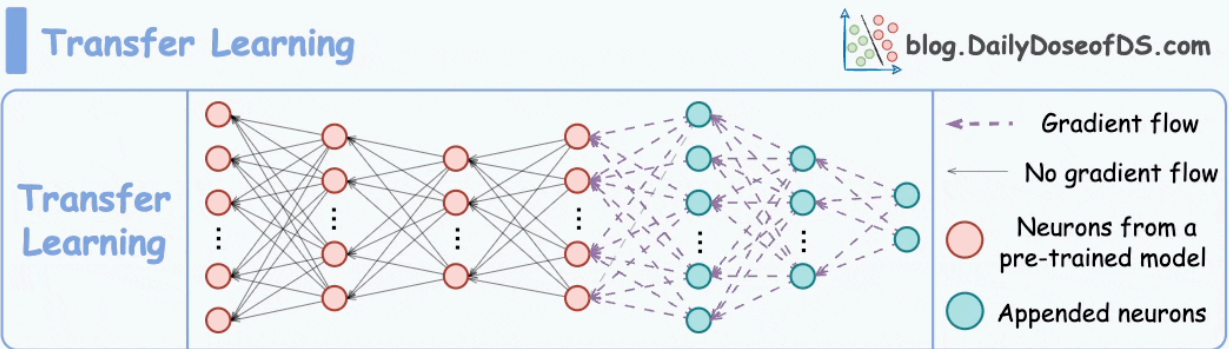
Transfer Learning, Fine-tuning, Multitask Learning and Federated Learning

Most ML models are trained independently without any interaction with other models. However, in the realm of real-world ML, there are many powerful learning techniques that rely on model interactions to improve performance.

The following image summarizes four such well-adopted and must-know training methodologies:



#1) Transfer learning



This is extremely useful when:

- The task of interest has less data.
- But a related task has abundant data.

This is how it works:

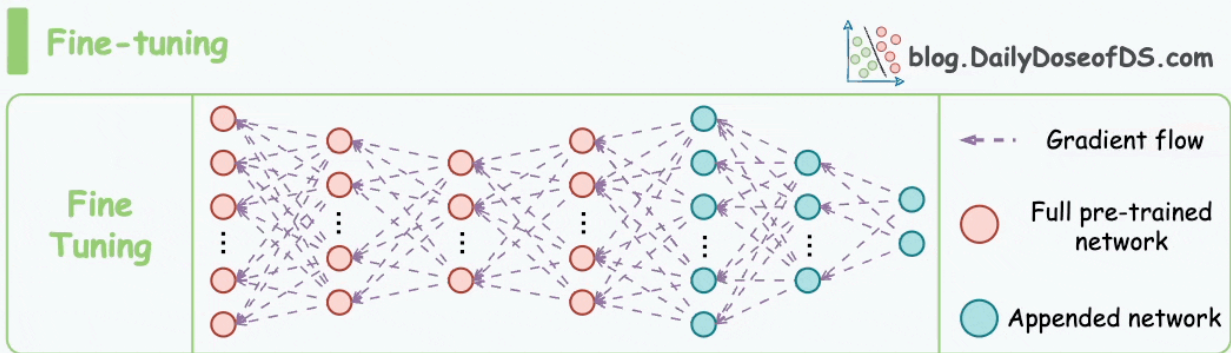
- Train a neural network model (base model) on the related task.
- Replace the last few layers on the base model with new layers.
- Train the network on the task of interest, but don't update the weights of the unreplaced layers during backpropagation.

By training a model on the related task first, we can capture the core patterns of the task of interest. Later, we can adjust the last few layers to capture task-specific behavior.

Another idea which is somewhat along these lines is knowledge distillation, which involves the “transfer” of knowledge. We will discuss it in the upcoming chapters.

Transfer learning is commonly used in many computer vision tasks.

#2) Fine-tuning

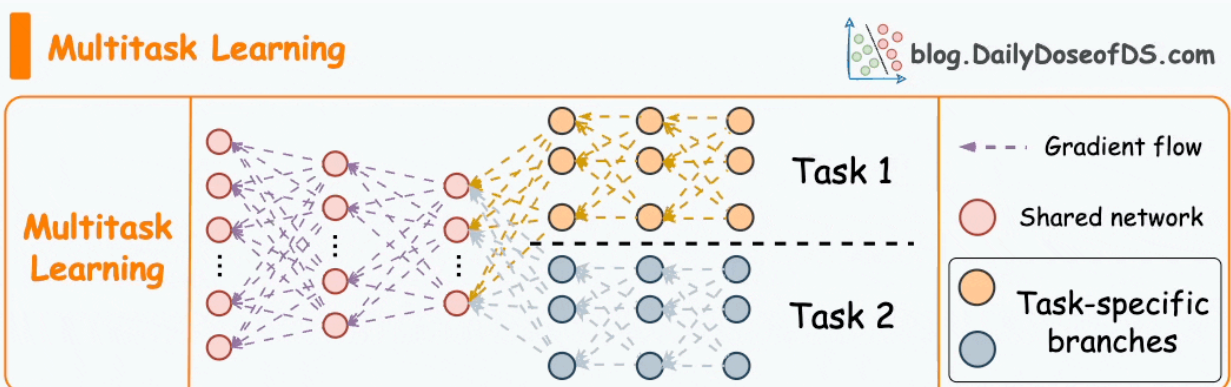


Fine-tuning involves updating the weights of some or all layers of the pre-trained model to adapt it to the new task.

The idea may appear similar to transfer learning, but in fine-tuning, we typically do not replace the last few layers of the pre-trained network.

Instead, the pretrained model itself is adjusted to the new data.

#3) Multi-task learning



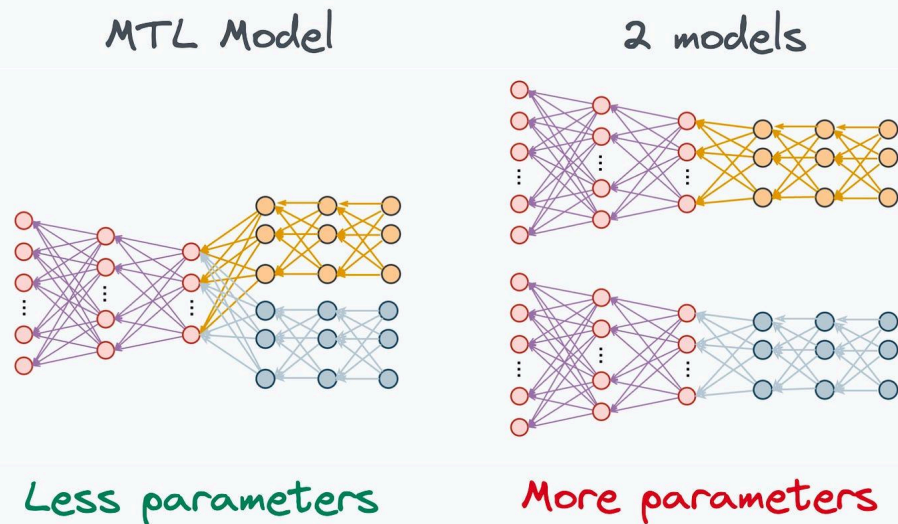
As the name suggests, a model is trained to perform multiple tasks simultaneously.

The model shares knowledge across tasks, aiming to improve generalization and performance on each task.

It can help in scenarios where tasks are related, or they can benefit from shared representations.

In fact, the motive for multi-task learning is not just to improve generalization.

We can also save compute power during training by having a shared layer and task-specific segments.



- Imagine training two models independently on related tasks.
- Now compare it to having a network with shared layers and then task-specific branches.

Option 2 will typically result in:

- Better generalization across all tasks.
- Less memory utilization to store model weights.
- Less resource utilization during training.

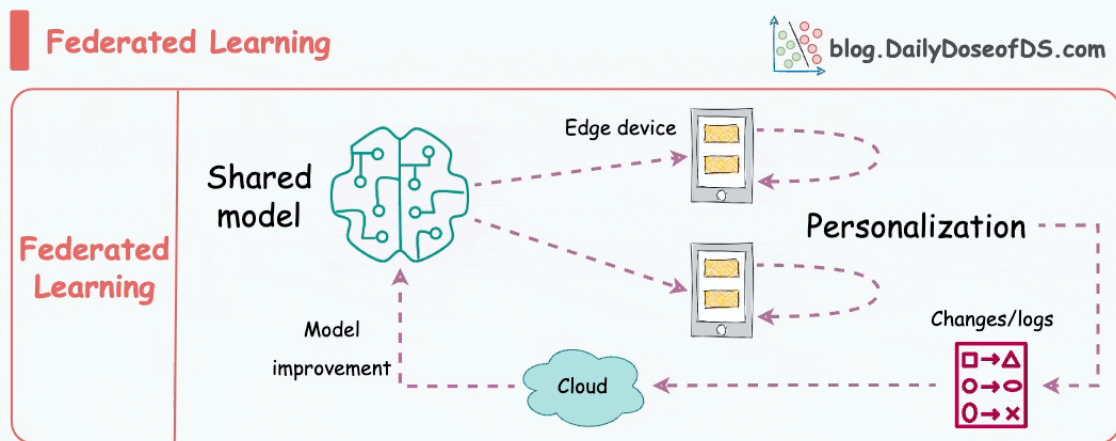
#4) Federated learning

Let's discuss it in the next chapter.

Introduction to Federated Learning

In my opinion, federated learning is among those very powerful ML techniques that is not given the true attention it deserves.

Here's a visual that depicts how it works:



Let's understand this topic in this chapter and why I consider this to be an immensely valuable skill to have.

The problem

Modern devices (like smartphones) have access to a wealth of data that can be suitable for ML models.

To get some perspective, consider the number of images you have on your phone right now, the number of keystrokes you press daily, etc.

That's plenty of data, isn't it?

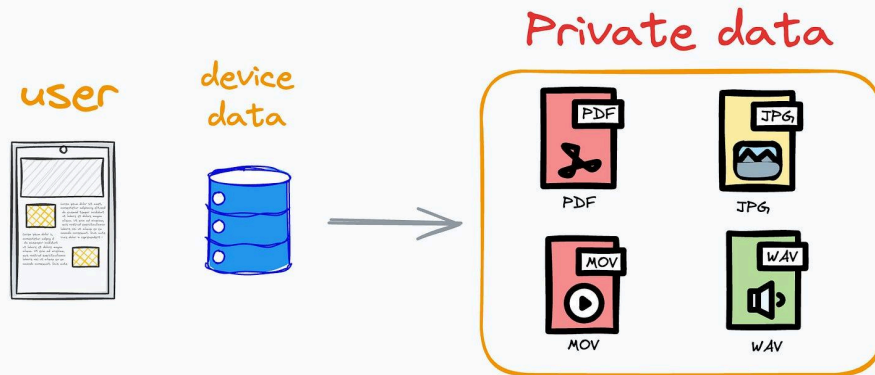
And this is just about one user — you.

But applications can have millions of users. The amount of data we can train ML models on is unfathomable.

So what is the problem here?

The problem is that almost all data available on modern devices is private.

- Images are private.
- Messages you send are private.
- Voice notes are private.



Being private, it is likely that it cannot be aggregated in a central location, as traditionally, ML models are always trained on centrally located datasets.

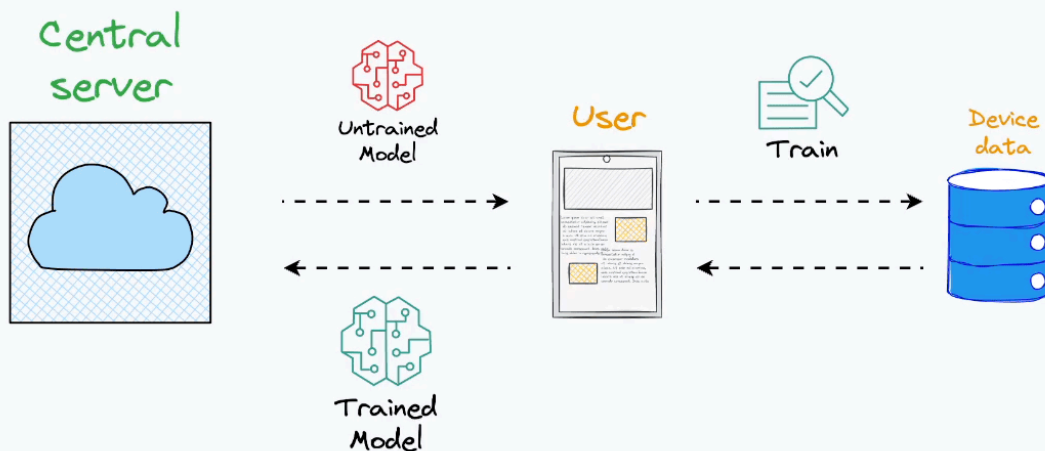
But this data is still valuable to us, isn't it?

We want to utilize it in some way.

The solution

Federated learning smartly addresses this challenge of training ML models on private data.

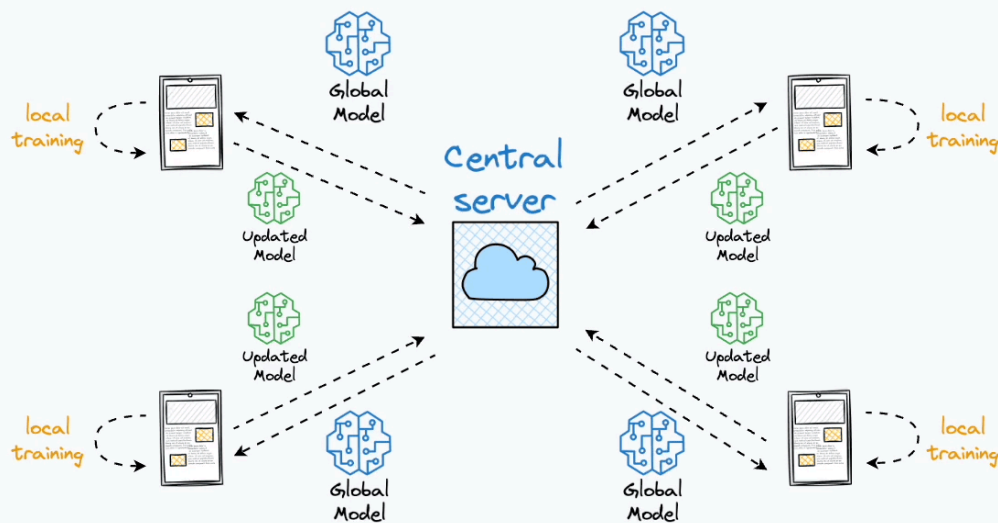
Here's the core idea:



- Instead of aggregating data on a central server, dispatch a model to an end device.
- Train the model on the user's private data on their device.
- Fetch the trained model back to the central server.
- Aggregate all models obtained from all end devices to form a complete model.

That's an innovative solution because each client possesses a local training dataset that remains exclusively on their device and is never uploaded to the server.

Yet, we still get to train a model on this private data.



Send a global model to the user's device, train a model on private data, and retrieve it back.

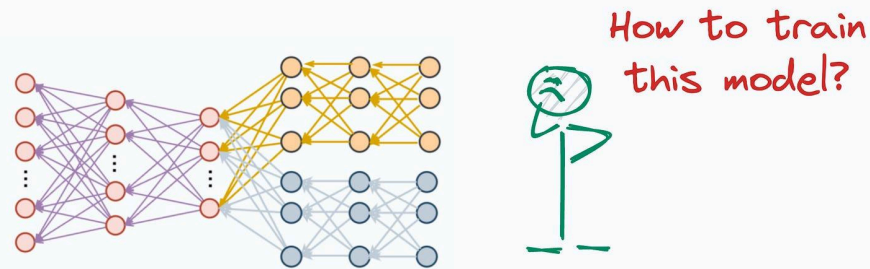
Furthermore, federated learning distributes most computation to a user's device.

As a result, the central server does not need the enormous computing that it would have demanded otherwise.

This is the core idea behind federated learning.

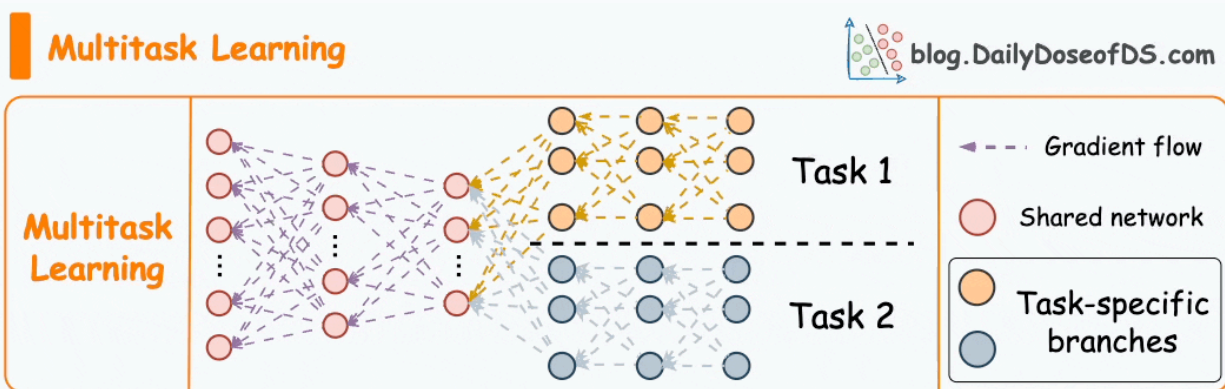
Building Multi-task Learning (MTL) Models

Most ML models are trained on one task. As a result, many struggle to intuitively understand how a model can be trained on multiple tasks simultaneously.



So let's discuss it in this chapter.

To reiterate, in MTL, the network has a few shared layers and task-specific segments. During backpropagation, gradients are accumulated from all branches, as depicted in the animation below:



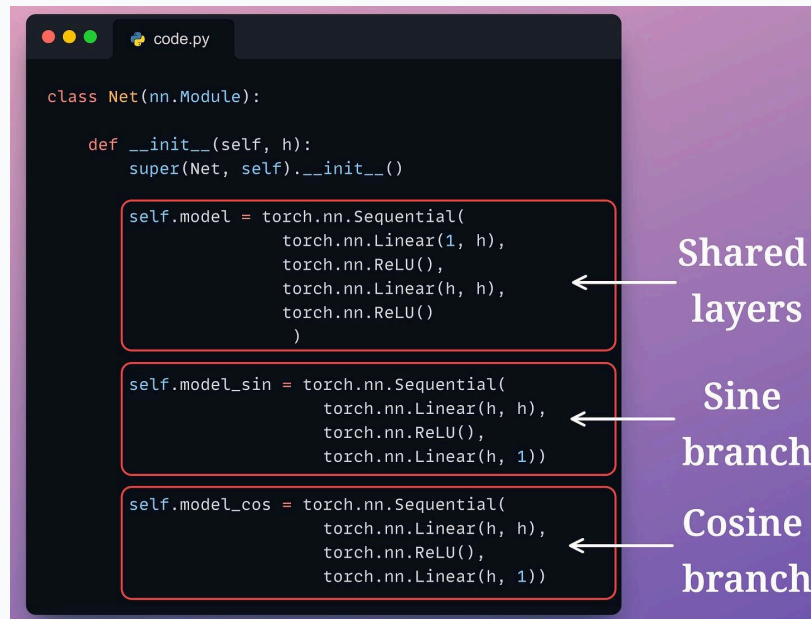
Let's take a simple example to understand its implementation.

Consider we want our model to take a real value (x) as input and generate two outputs:

- $\sin(x)$
- $\cos(x)$

This can be formulated as an MTL problem.

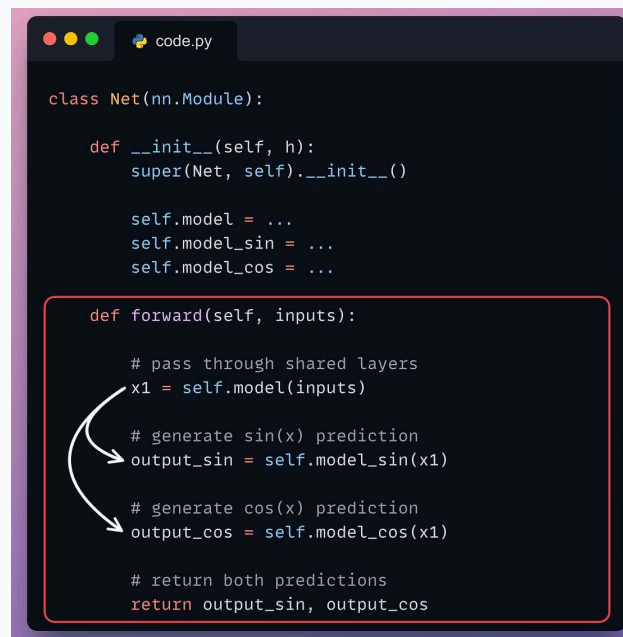
First, we define our model class using PyTorch.



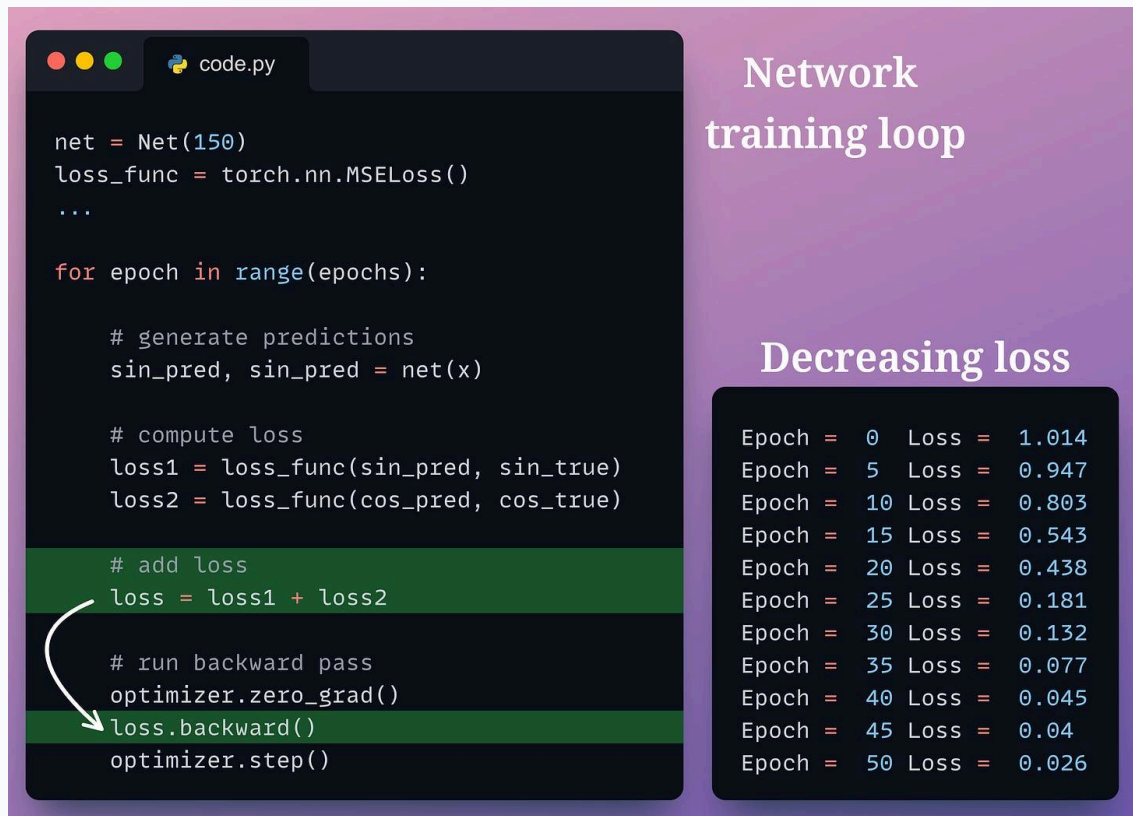
- We have some fully connected layers in `self.model` → These are the shared layers.
- Furthermore, we have the output-specific layers to predict $\sin(x)$ and $\cos(x)$.

Next, let's define the forward pass in the class above:

- First, we pass the input through the shared layers (`self.model`).
- The output of the shared layers is passed through the sin and cos branches.
- We return the output from both branches.



We are almost done. The final part of this implementation is to train the model. Let's use mean squared error as the loss function. The training loop is implemented below:



- We pass the input data through the model.
- It returns two outputs, one from each segment of the network.
- We compute the branch-specific loss values (loss1 and loss2) using true predictions.
- We add the two loss values to get the total loss for the network.
- Finally, we run the backward pass.

With this, we have trained our MTL model. Also, we get a decreasing loss, which depicts that the model is being trained.

And that's how we train an MTL model. You can extend the same idea to build any MTL model of your choice.

Do remember that building an MTL model on unrelated tasks will not produce good results.

Thus, “task-relatedness” is a critical component of all MTL models because of the shared layers. Also, it is NOT necessary that every task must equally contribute to the entire network’s loss.

We may assign weights to each task as well, as depicted below:

$$\text{loss} = \overset{\text{weighted loss}}{c1 * \text{loss_task1} + c2 * \text{loss_task2}}$$

The weights could be based on task importance.

Or...

At times, I also use dynamic task weights, which could be inversely proportional to the validation accuracy achieved on that task.

$$c1 \propto \frac{1}{\text{accuracy_task1}} \quad c2 \propto \frac{1}{\text{accuracy_task2}}$$

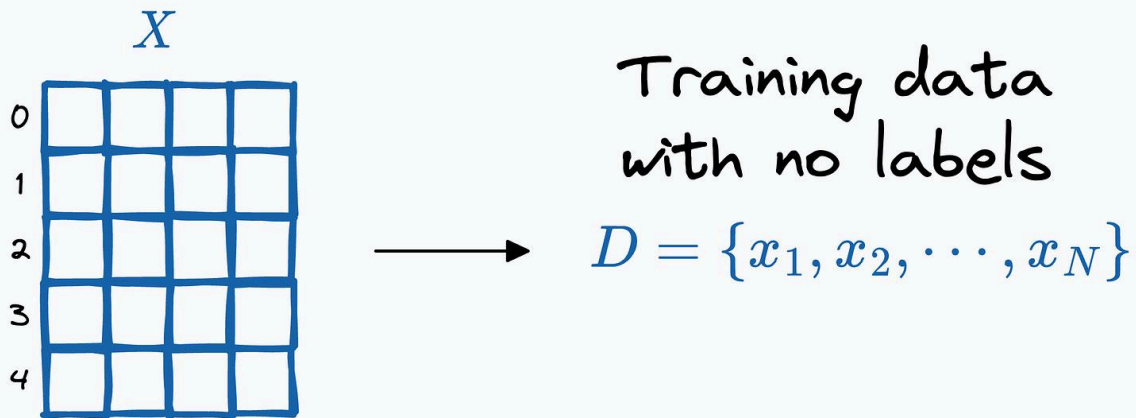
My rationale behind this technique is that in an MTL setting, some tasks can be easy while others can be difficult.

If the model achieves high accuracy on one task during training, we can safely reduce its loss contribution so that the model focuses more on the second task.

You can download the notebook for this chapter here: <https://bit.ly/3ztY5hy>.

Active Learning

There's not much we can do to build a supervised system when the data we begin with is unlabeled.



Using unsupervised techniques (if they fit the task) can be a solution, but supervised systems are typically on par with unsupervised ones.

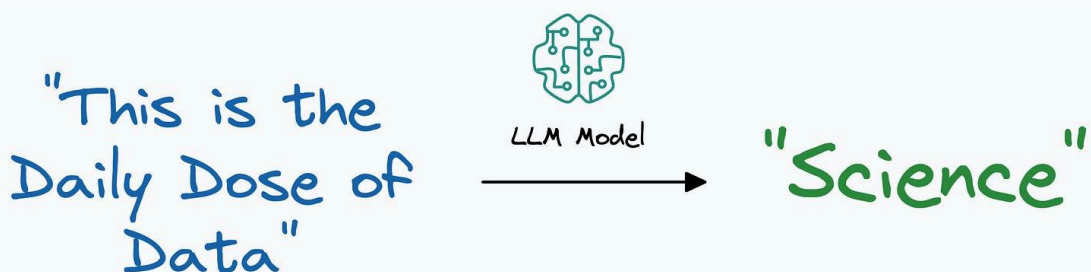
Another way, if feasible, is to rely on self-supervised learning.

Self-supervised learning is when we have an unlabeled dataset (say text data), but we somehow figure out a way to build a supervised learning model out of it.

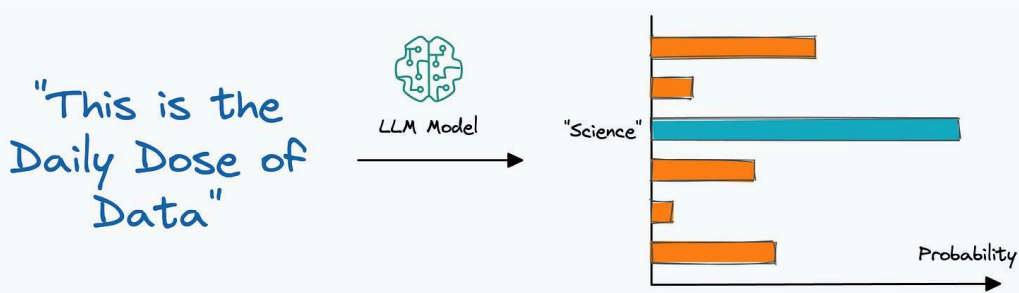
This becomes possible due to the inherent nature of the task.

Consider an LLM, for instance.

In a nutshell, its core objective is to predict the next token based on previously predicted tokens (or the given context).

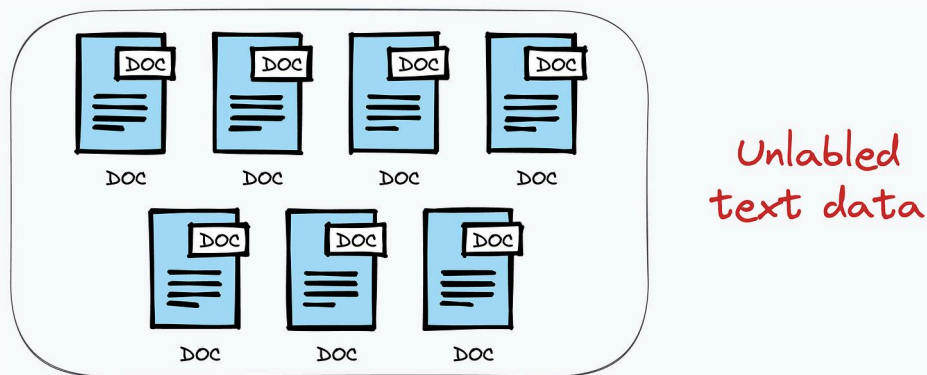


This is a classification task, and the labels are tokens.



But text data is raw. It has no labels.

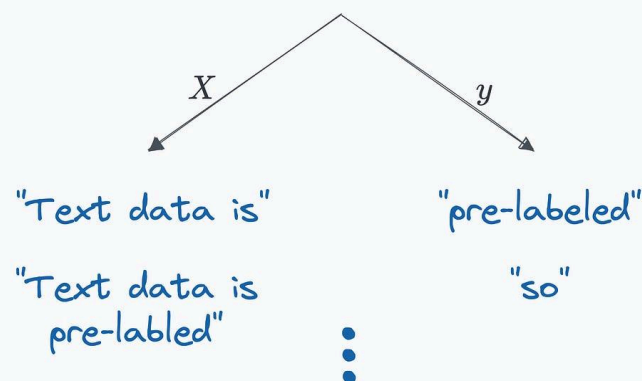
Then how did we train this classification task?



Self-supervised techniques solve this problem.

Due to the inherent nature of the task (next-token prediction, to be specific), every piece of raw text data is already self-labeled.

"Text data is pre-labeled so it does not need any labeling"



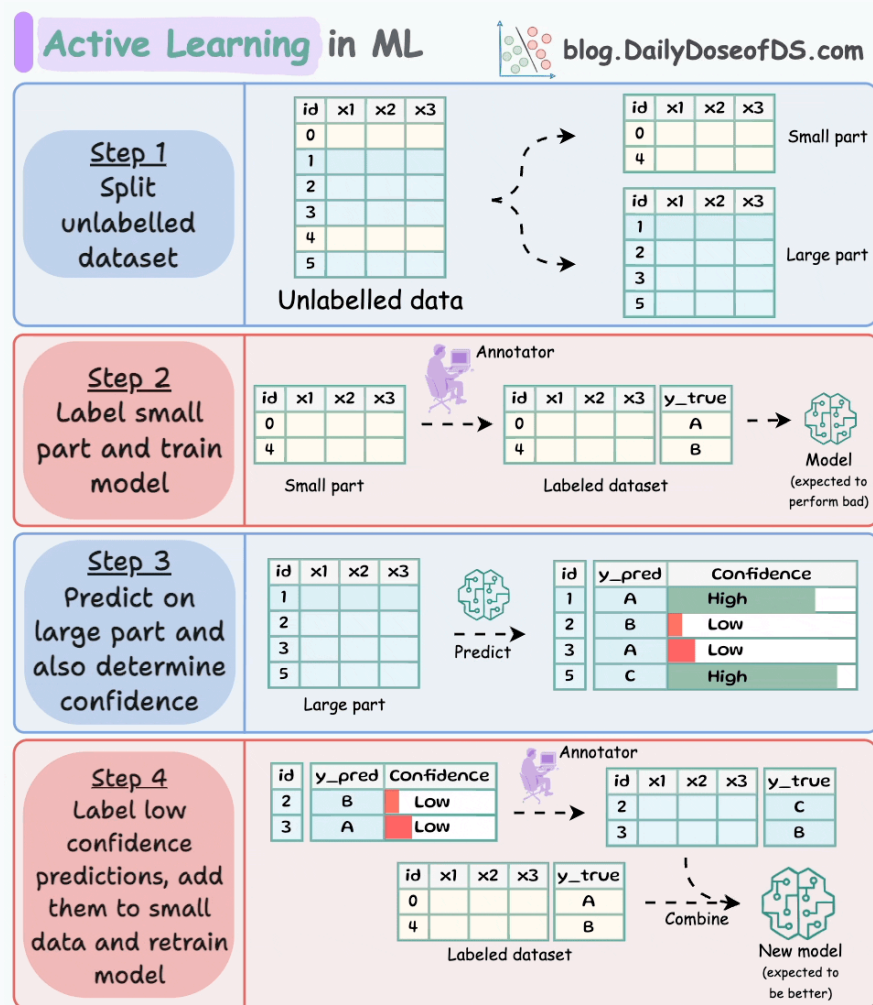
The model is only supposed to learn the mapping from previous tokens to the next token.

This is called self-supervised learning, which is quite promising, but it has limited applicability, largely depending on the task.

At this stage, the only possibility one notices is annotating the dataset. However, data annotation is difficult, expensive, time-consuming, and tedious.

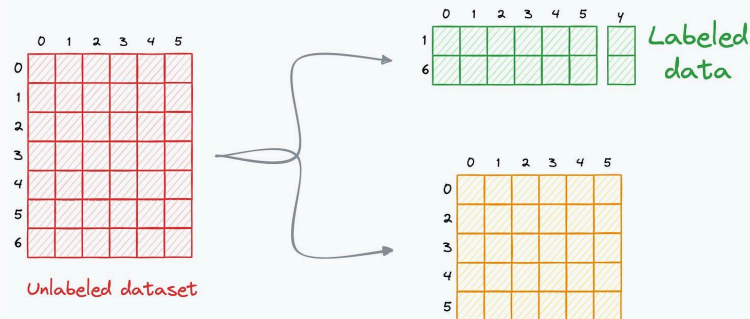
Active learning is a relatively easy, inexpensive, quick, and interesting way to address this.

As the name suggests, the idea is to build the model with active human feedback on examples it is struggling with. The visual below summarizes this:



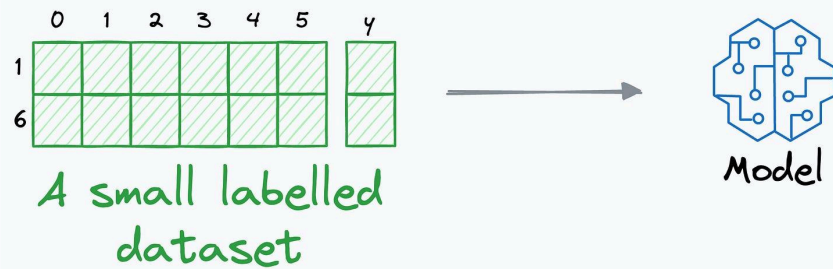
Let's get into the details.

We begin by manually labeling a tiny percentage of the dataset.

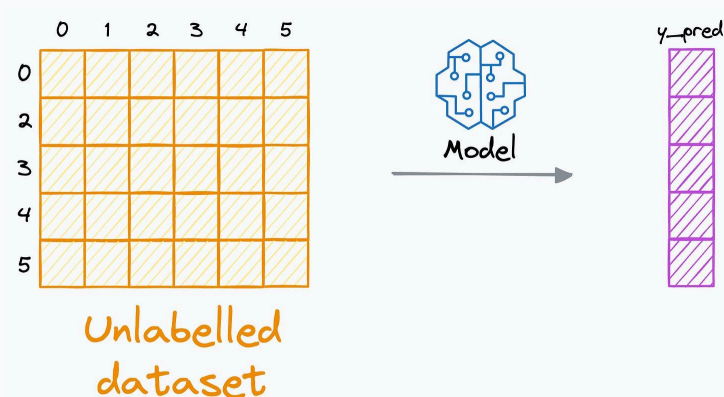


While there's no rule on how much data should be labeled, I have used active learning (successfully) while labeling as low as ~1% of the dataset, so try something in that range.

Next, build a model on this small labeled dataset.



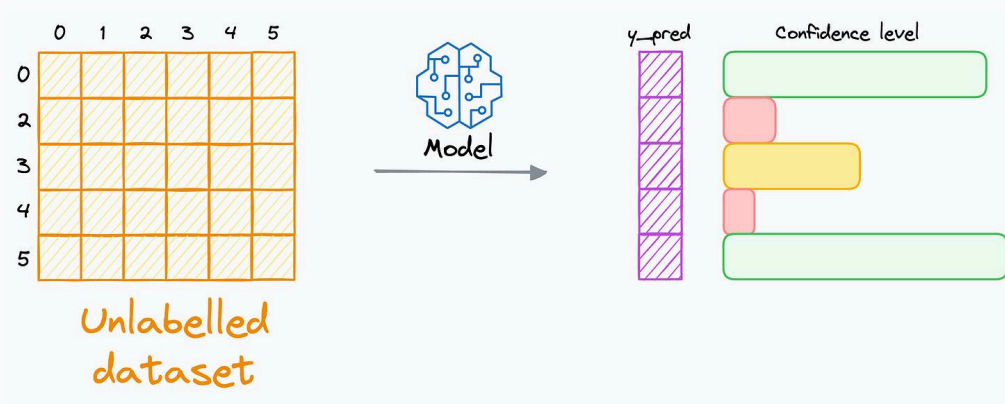
Of course, this won't be a perfect model, but that's okay. Next, generate predictions on the dataset we did not label:



It's obvious that we cannot determine if these predictions are correct as we do not have any labels.

That's why we need to be a bit selective with the type of model we choose.

More specifically, we need a model that, either implicitly or explicitly, can also provide a confidence level with its predictions.

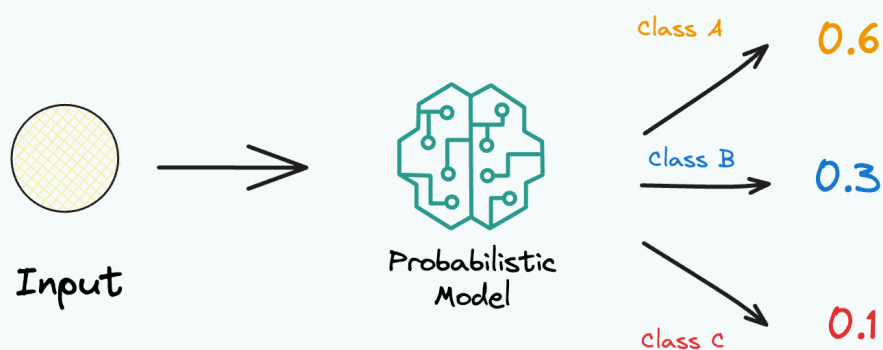


As the name suggests, a confidence level reflects the model's confidence in generating a prediction.

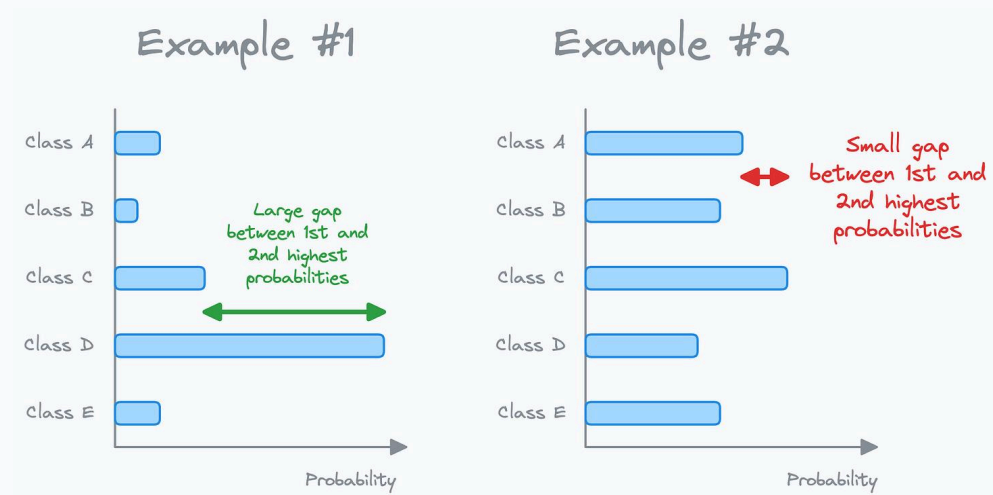
If a model could speak, it would be like:

- I am predicting a “cat” and am 95% confident about my prediction.
- I am predicting a “cat” and am 5% confident about my prediction.
- And so on...

Probabilistic models (ones that provide a probabilistic estimate for each class) are typically a good fit here.



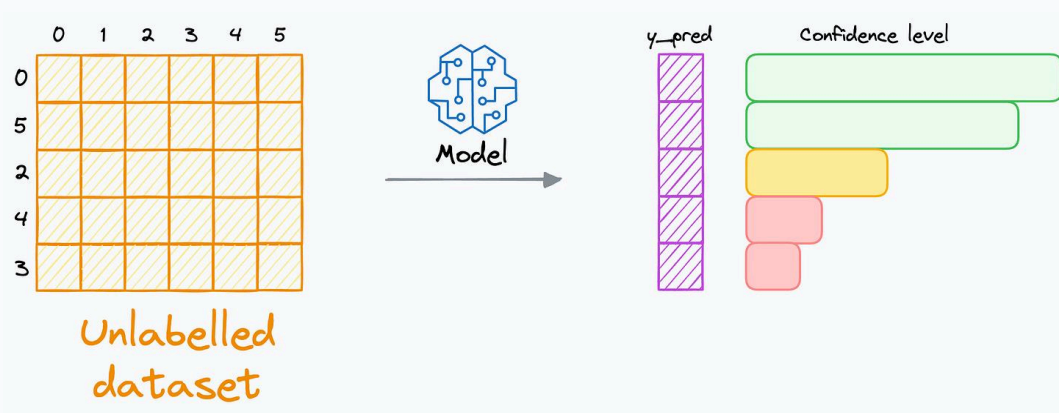
This is because one can determine a proxy for confidence level from probabilistic outputs.



In the above two examples, consider the gap between 1st and 2nd highest probabilities:

- In example #1, the gap is large. This can indicate that the model is quite confident in its prediction.
- In example #2, the gap is small. This can indicate that the model is NOT quite confident in its prediction.

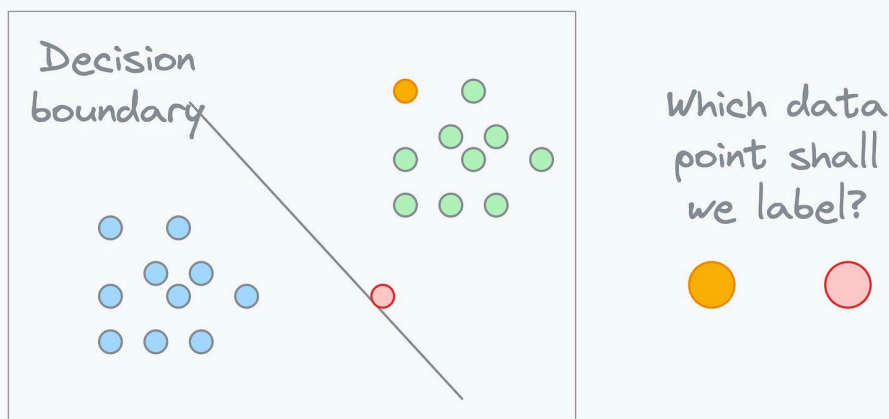
Now, go back to the predictions generated above and rank them in order of confidence:



In the above image:

- The model is already quite confident with the first two instances. There's no point checking those.
- Instead, it would be best if we (the human) annotate the instances with which it is least confident.

To get some more perspective, consider the image below. Logically speaking, which data point's human label will provide more information to the model? I know you already know the answer.



Thus, in the next step, we provide our human label to the low-confidence predictions and feed it back to the model with the previously labeled dataset:



Repeat this a few times and stop when you are satisfied with the performance.

In my experience, active learning has always been an immensely time-saving approach to building supervised models on unlabeled datasets.

The only thing that you have to be careful about is generating confidence measures.

If you mess this up, it will affect every subsequent training step.

There's one more thing I like to do when using active learning.

While combining the low-confidence data with the seed data, we can also use the high-confidence data. The labels would be the model's predictions.

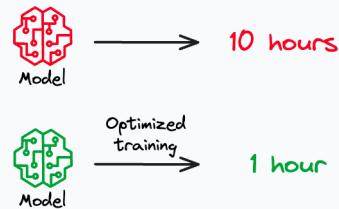


This variant of active learning is called cooperative learning.

Run-time and Memory Optimization

Momentum

As we progress towards building larger and larger models, every bit of possible optimization becomes crucial.



And, of course, there are various ways to speed up model training, like:

- Batch processing
- Leverage distributed training using frameworks like PySpark MLlib.
- Use better Hyperparameter Optimization, like Bayesian Optimization, which we will discuss in [this chapter](#).
- and many other techniques.

Momentum is another reliable and effective technique to speed up model training. While Momentum is pretty popular, many people struggle to intuitively understand how it works and why it is effective. Let's understand in this chapter.

Issues with Gradient Descent

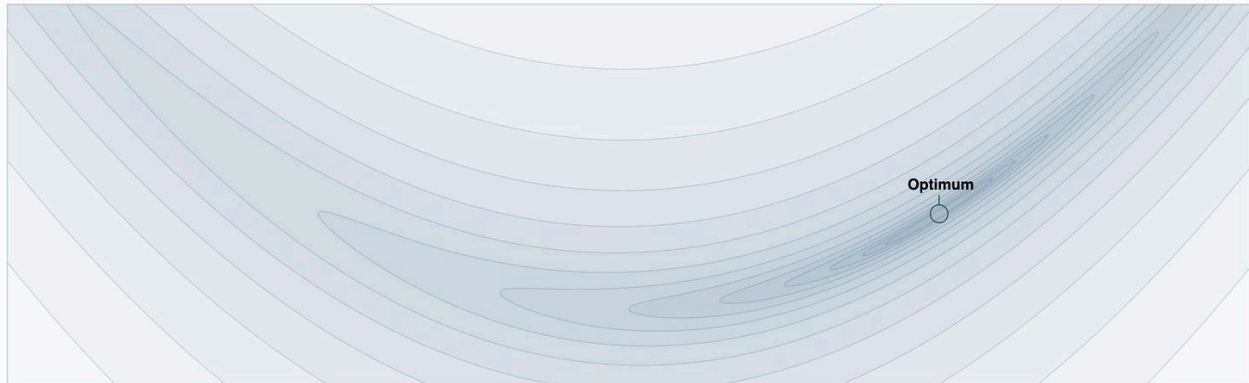
In gradient descent, every parameter update solely depends on the current gradient. This is clear from the gradient weight update rule shown below:

$$\theta^t = \theta^{t-1} - \alpha \frac{\delta J}{\delta \theta} \quad \leftarrow \text{Only dependent on current gradient}$$

As a result, we end up having many unwanted oscillations during the optimization process.

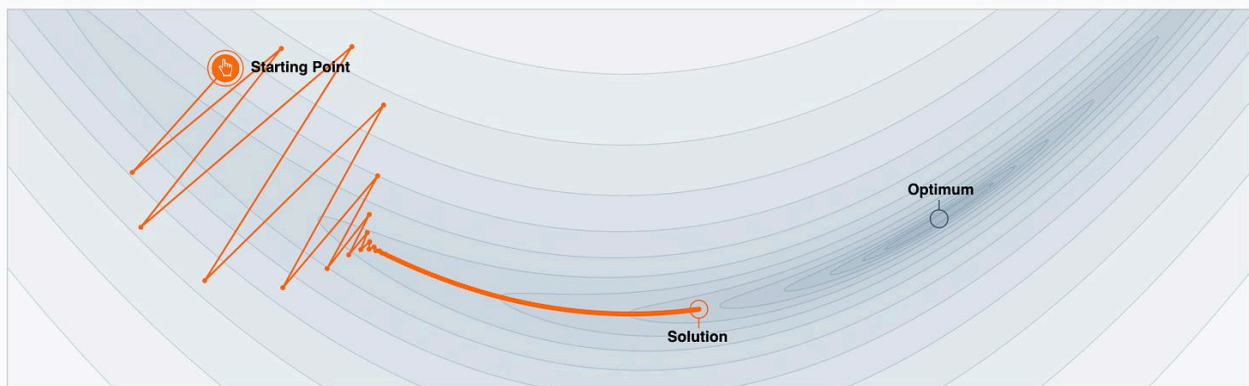
Let's understand this more visually.

Imagine this is the loss function contour plot, and the optimal location (parameter configuration where the loss function is minimum) is marked here:



Simply put, this plot illustrates how gradient descent moves towards the optimal solution. At each iteration, the algorithm calculates the gradient of the loss function at the current parameter values and updates the weights.

This is depicted below:



Notice two things here:

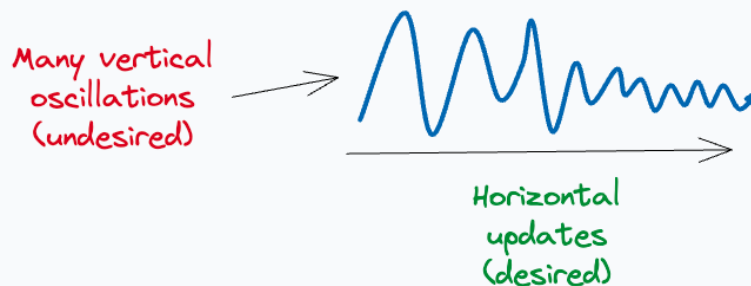
- It unnecessarily oscillates vertically.
- It ends up at the non-optimal solution after some epochs.

Ideally, we would have expected our weight updates to look this:



- It must have taken longer steps in the horizontal direction...
- ...and smaller vertical steps because a movement in this direction is unnecessary.

This idea is also depicted below:



How Momentum solves this problem?

Momentum-based optimization slightly modifies the update rule of gradient descent. More specifically, it also considers a moving average of past gradients:

$$\theta^t = \theta^{t-1} - \alpha \frac{\delta J}{\delta \theta} + \beta \cdot F(\theta^{t-1}, \theta^{t-2}, \dots)$$

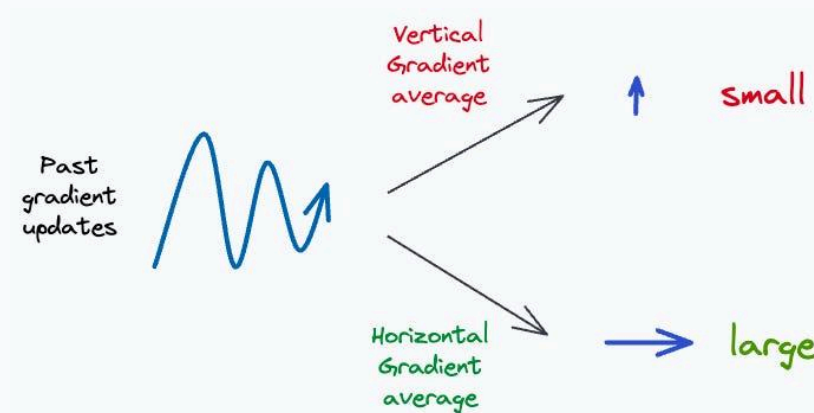
↑
↑

dependent on current gradient
dependent on previous updates

This helps us handle the unnecessary vertical oscillations we saw earlier.

How?

As Momentum considers a moving average of past gradients, so if the recent gradient update trajectory looks as shown in the following image, then it is clear that its average in the vertical direction will be very low while that in the horizontal direction will be large (which is precisely what we want):



As this moving average gets added to the gradient updates, it helps the optimization algorithm take larger steps in the desired direction.

$$\theta^t = \theta^{t-1} - \alpha \frac{\delta J}{\delta \theta} + \beta \cdot \boxed{F(\theta^{t-1}, \theta^{t-2}, \dots)}$$

↑
↑

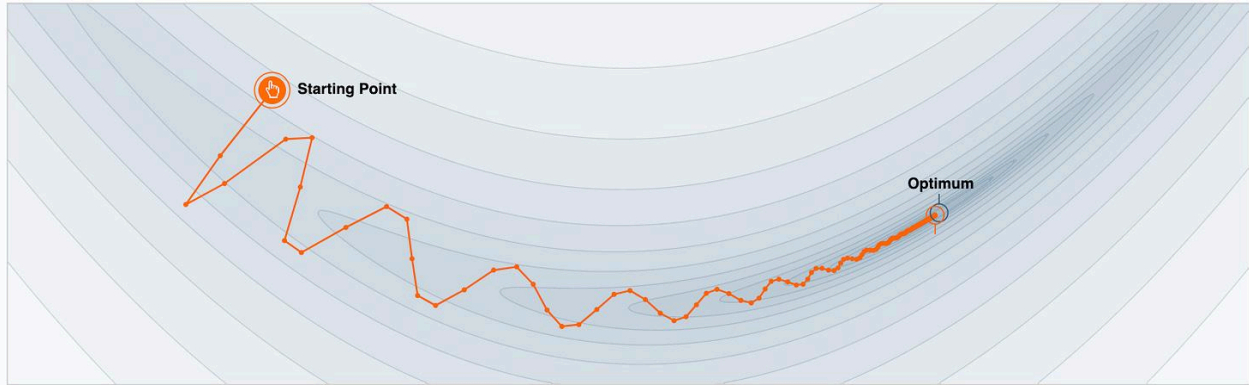
dependent on
current gradient
dependent on
previous updates

This way, we can:

- Smoothen the optimization trajectory.

- Reduce unnecessary oscillations in parameter updates, which also speeds up training.

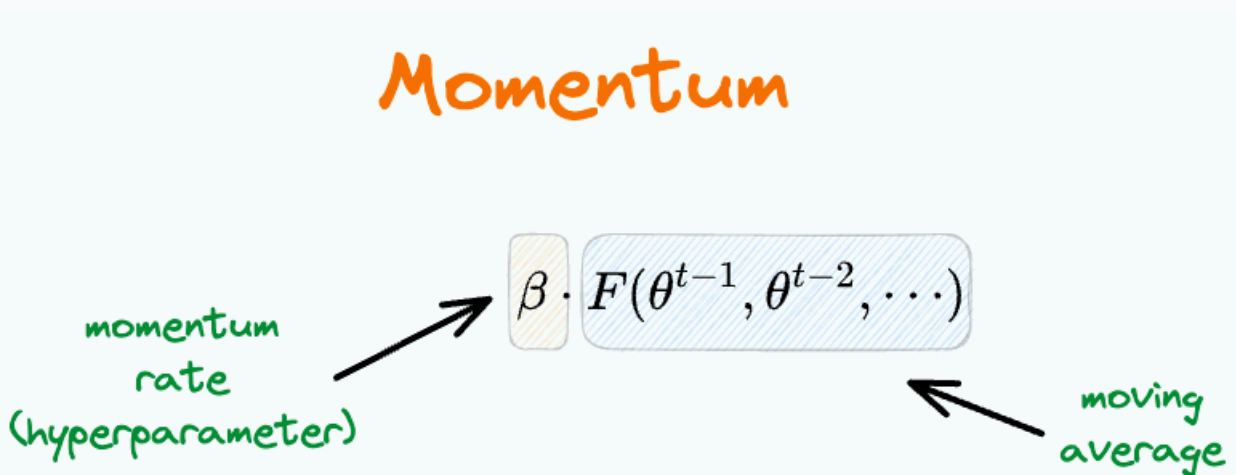
This is also evident from the image below:



This time, the gradient update trajectory shows much smaller oscillations in the vertical direction, and it also manages to reach an optimum under the same number of epochs as earlier.

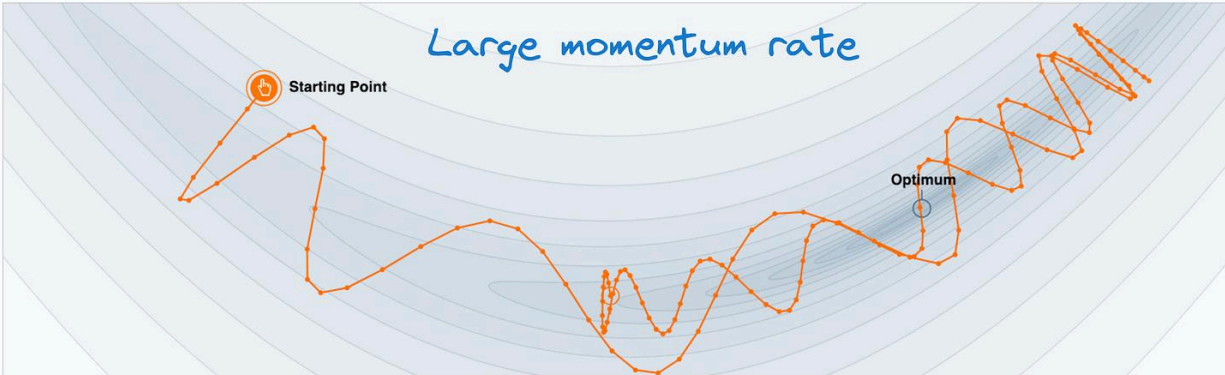
This is the core idea behind Momentum and how it works.

Of course, Momentum does introduce another hyperparameter (Momentum rate) in the model, which should be tuned appropriately like any other hyperparameter:

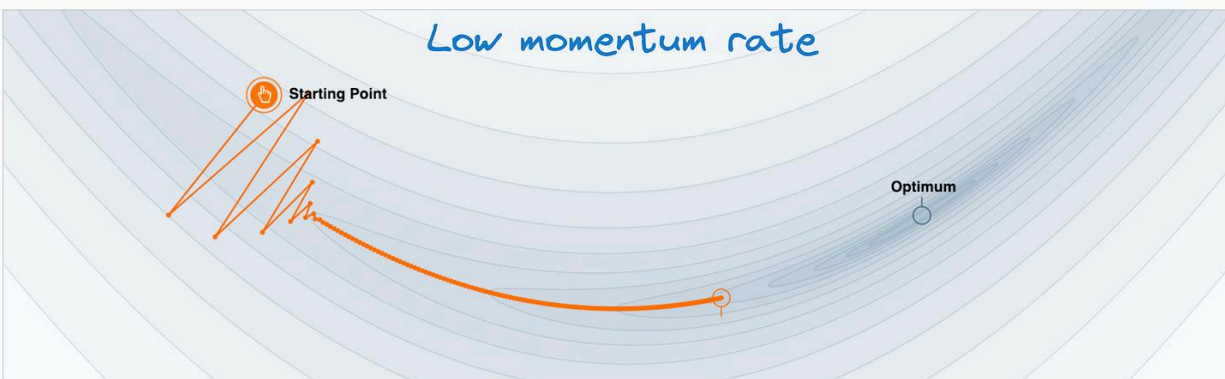


For instance, considering the 2D contours we discussed above:

- Setting an extremely large value of Momentum rate will significantly expedite gradient update in the horizontal direction. This may lead to overshooting the minima, as depicted below:



- What's more, setting an extremely small value of Momentum will slow down the optimal gradient update, defeating the whole purpose of Momentum.



If you want to have a more hands-on experience, check out this tool:

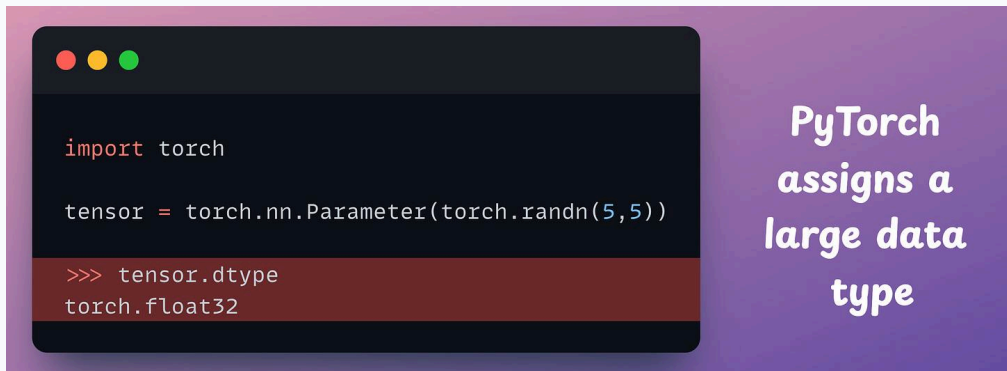
<https://bit.ly/4cOrJN1>.

Mixed Precision Training

Context

Typical deep learning libraries are really conservative when it comes to assigning data types.

The data type assigned by default is usually 64-bit or 32-bit, when there is also scope for 16-bit, for instance. This is also evident from the code below:



```
import torch

tensor = torch.nn.Parameter(torch.randn(5,5))

>>> tensor.dtype
torch.float32
```

**PyTorch
assigns a
large data
type**

As a result, we are not entirely optimal at efficiently allocating memory. Of course, this is done to ensure better precision in representing information.



Lowest precision

```
a = np.array([0.123456789123456789], dtype=np.float16 )
>>> print("16-bits: ", a[0])
"16-bits:  0.1235"
```

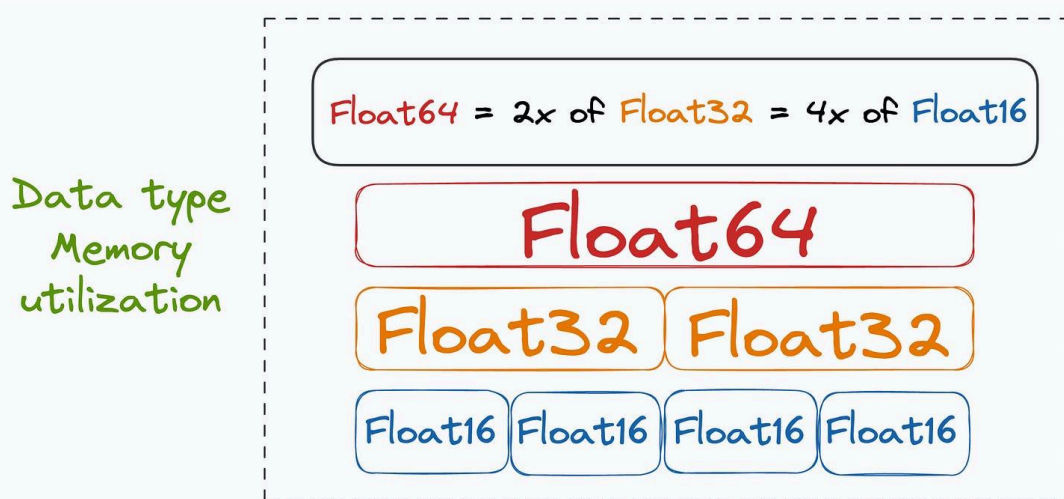
Intermediate precision

```
b = np.array([0.123456789123456789], dtype=np.float32 )
>>> print("32-bits: ", b[0])
"32-bits:  0.12345679"
```

Highest precision

```
c = np.array([0.123456789123456789], dtype=np.float64 )
>>> print("64bit: ", c[0])
"64-bits:  0.12345678912345678"
```

However, this precision always comes at the cost of additional memory utilization, which may not be desired in all situations.



In fact, it is also observed that many tensor operations, especially matrix multiplication, are much faster when we operate under smaller precision data types than larger ones, as demonstrated below:

Float 32 matrix multiplication

```
matrixA = torch.randn(1000, 1000, dtype=torch.float32, device=device)
matrixB = torch.randn(1000, 1000, dtype=torch.float32, device=device)
```

```
%timeit torch.matmul(matrixA, matrixB)
```

658 μ s $\pm$ 12.4 μ s per loop

Float 16 matrix multiplication

```
matrixA = torch.randn(1000, 1000, dtype=torch.float16, device=device)
matrixB = torch.randn(1000, 1000, dtype=torch.float16, device=device)
```

```
%timeit torch.matmul(matrixA, matrixB)
```

111 μ s $\pm$ 4.07 μ s per loop

6x Faster

Moreover, since `float16` is only half the size of `float32`, its usage reduces the memory required to train the network. This also allows us to train larger models, train on larger mini-batches (resulting in even more speedup), etc.

Mixed precision training is a pretty reliable and widely adopted technique in the industry to achieve this.

As the name suggests, the idea is to employ lower precision `float16` (wherever feasible, like in convolutions and matrix multiplications) along with `float32` — that is why the name “mixed precision.”

This is a list of some models I found that were trained using mixed precision:

Image Classification	Detection / Segmentation	Generative Models (Images)	Language Modeling
AlexNet	DeepLab	DLSS	BERT
DenseNet	Faster R-CNN	Vid2vid	GPT
Inception	Mask R-CNN	GauGAN	TrellisNet
MobileNet	SSD	Partial Image Inpainting	Gated Convolutions
EfficientNet	NVIDIA Automotive	Progress GAN	BigLSTM/mLSTM
ResNet	RetinaNet	Pix2Pix	RoBERTa
ResNeXt	UNET		Transformer XL
ShuffleNet	DETR		
SqueezeNet			
VGG			
Xception			
Dilated ResNet			
Stacked U-Net			

Recommendation
DeepRecommender
DLRM
NCF

Speech
Deep Speech 2
Jasper
Tacotron
Wave2vec
WaveNet
WaveGlow

Translation
Convolutional Seq2Seq
Dynamic Convolutions
GNMT (RNN)
Levenshtein Transformer
Transformer (Self-Attention)

It's pretty clear that mixed precision training is much more popularly used, but we don't get to hear about it often.

Before we get into the technical details...

From the above discussion, it must be clear that as we use a low-precision data type (`float16`), we might unknowingly introduce some numerical inconsistencies and inaccuracies.

To avoid them, there are some best practices for mixed precision training that I want to talk about next, along with the code.

Mixed precision training in PyTorch and Best Practices

Leveraging mixed precision training in PyTorch requires a few modifications in the existing network training implementation. Consider this is our current PyTorch model training implementation:

```
net = MyModel(in_size, out_size, num_layers)
opt = torch.optim.SGD(net.parameters(), lr=0.001)
loss_fn = torch.nn.MSELoss()

for epoch in range(epochs):

    for inputs, target in zip(data, targets):

        # generate prediction
        output = net(inputs)

        # compute loss
        loss = loss_fn(output, target)

        # compute gradients
        loss.backward()

        # update weights
        opt.step()

        # zero gradients
        opt.zero_grad()
```

**PyTorch
Model training
loop**

The first thing we introduce here is a scaler object that will scale the loss value:

```
net = MyModel(in_size, out_size, num_layers)
opt = torch.optim.SGD(net.parameters(), lr=0.001)
loss_fn = torch.nn.MSELoss()

scaler = torch.cuda.amp.GradScaler()

...
```

**Loss
scaler**

We do this because, at times, the original loss value can be so low, that we might not be able to compute gradients in `float16` with full precision. Such situations may not produce any update to the model's weights.

$$w_{t+1} = w_t - \alpha \cdot J(w)$$

No update takes place

Scaling the loss to a higher numerical range ensures that even small gradients can contribute to the weight updates.

But these minute gradients can only be accommodated into the weight matrix when the weight matrix itself is represented in high precision, i.e., `float32`. Thus, as a conservative measure, we tend to keep the weights in `float32`.

That said, the loss scaling step is not entirely necessary because, in my experience, these little updates typically appear towards the end stages of the model training. Thus, it can be fair to assume that small updates may not drastically impact the model performance. But don't take this as a definite conclusion, so it's something that I want you to validate when you use mixed precision training.

Moving on, as the weights (which are matrices) are represented in `float32`, we can not expect the speedup from representing them in `float16`, if they remain this way:

```

Float 32 matrix multiplication

matrixA = torch.randn(1000, 1000, dtype=torch.float32, device=device)
matrixB = torch.randn(1000, 1000, dtype=torch.float32, device=device)

%timeit torch.matmul(matrixA, matrixB)
658 µs ± 12.4 µs per loop

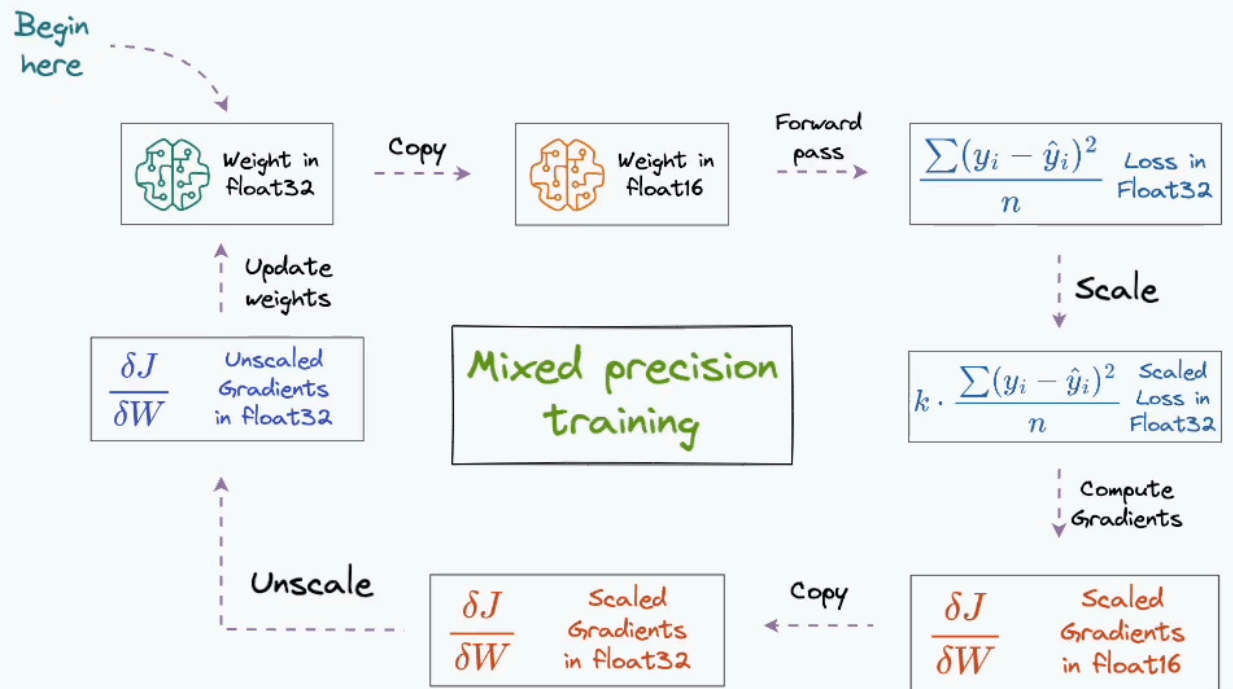
Float 16 matrix multiplication

matrixA = torch.randn(1000, 1000, dtype=torch.float16, device=device)
matrixB = torch.randn(1000, 1000, dtype=torch.float16, device=device)

%timeit torch.matmul(matrixA, matrixB)
111 µs ± 4.07 µs per loop
6x Faster

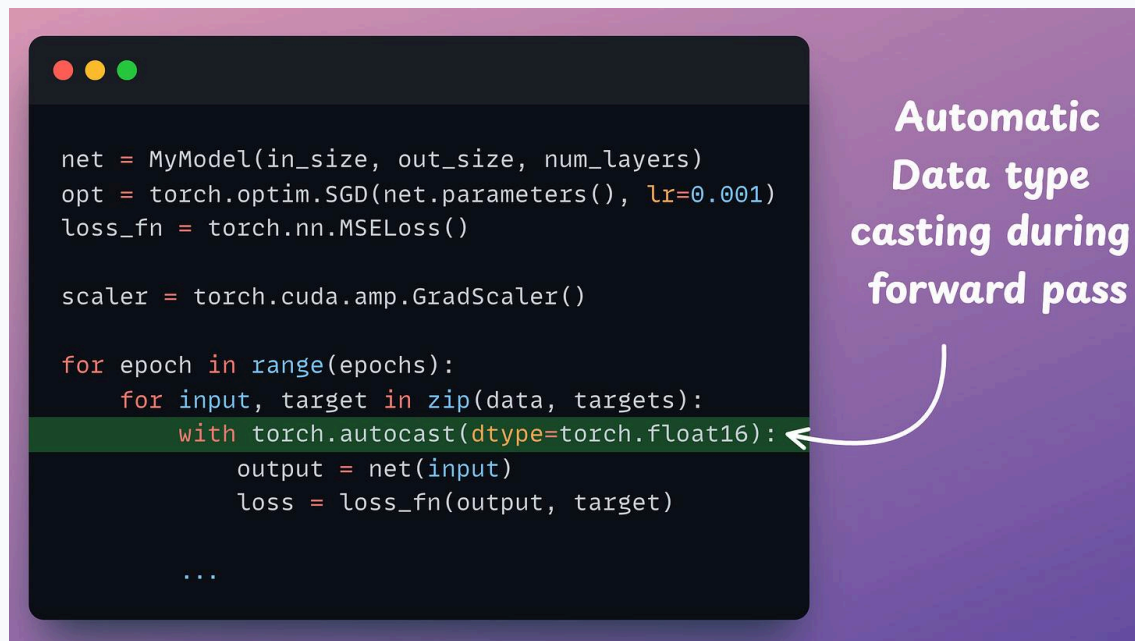
```

To leverage these float16-based speedups, here are the steps we follow:

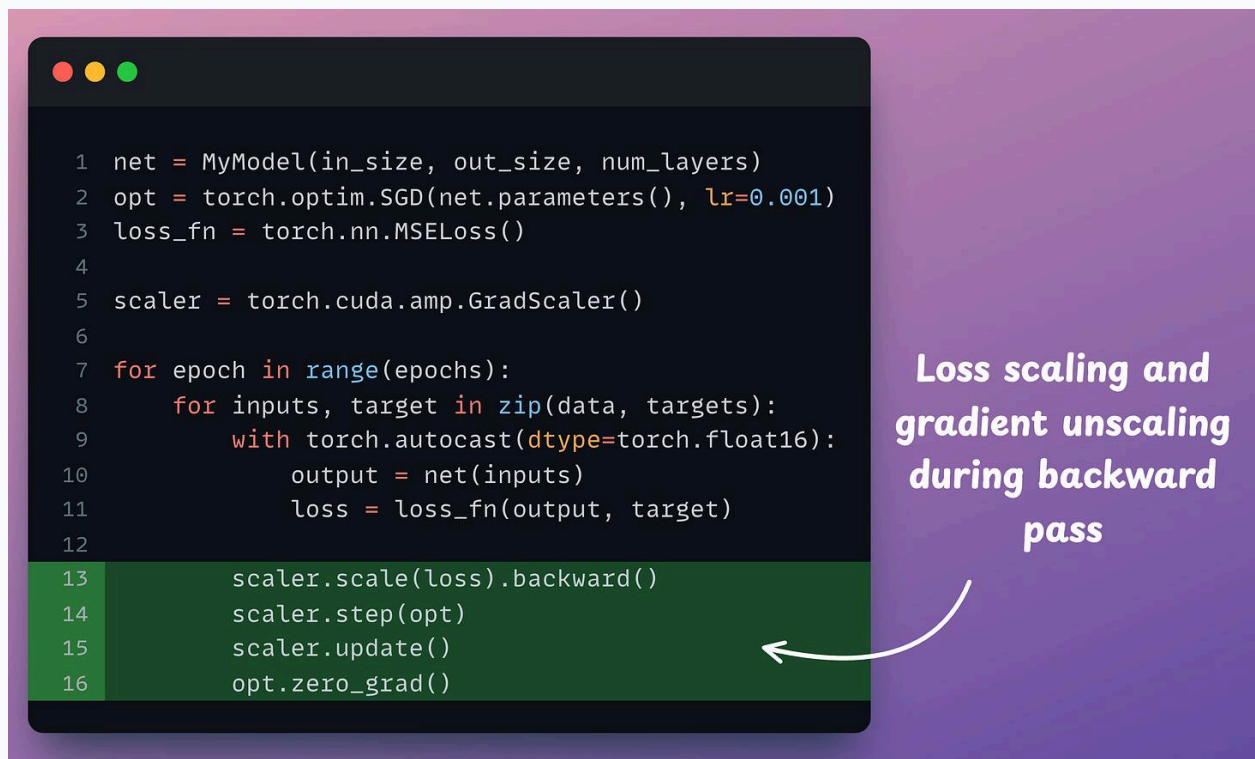


1. We make a `float16` copy of weights during the forward pass.
2. Next, we compute the loss value in `float32` and scale it to have more precision in gradients, which works in `float16`.
 - a. The reason we compute gradients in `float16` is because, like forward pass, gradient computations also involve matrix multiplications.
 - b. Thus, keeping them in `float16` can provide additional speedup.
3. Once we have computed the gradients in `float16`, the heavy matrix multiplication operations have been completed. Now, all we need to do is update the original weight matrix, which is in `float32`.
4. Thus, we make a `float32` copy of the above gradients, remove the scale we applied in Step 2, and update the `float32` weights.
5. Done!

The mixed-precision settings in the forward pass are carried out by the `torch.autocast()` context manager:



Now, it's time to handle the backward pass.



- Line 13 → `scaler.scale(loss).backward()`: The scaler object scales the loss value and `backward()` is called to compute the gradients.
- Line 14 → `scaler.step(opt)`: Unscale gradients and update weights.
- Line 15 → `scaler.update()`: Update the scale for the next iteration.
- Line 16 → `opt.zero_grad()`: Zero gradients.

Done!

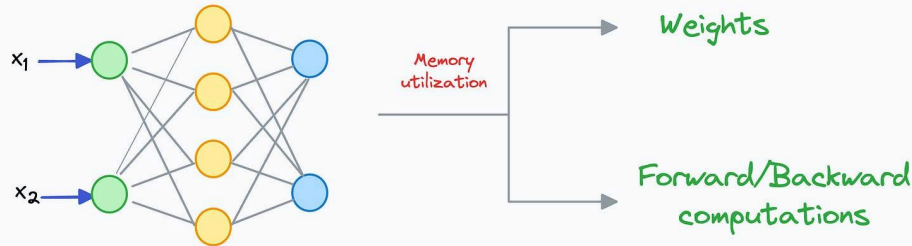
The efficacy of mixed precision scaling over traditional training is evident from the image below:

Usual training loop	Mixed precision training
<pre>[4] net = make_model(in_size, out_size, num_layers) opt = torch.optim.SGD(net.parameters(), lr=0.001) start_timer() for epoch in range(epochs): for input, target in zip(data, targets): output = net(input) loss = loss_fn(output, target) loss.backward() opt.step() opt.zero_grad() end_timer_and_print("Default precision:")</pre>	<pre>[5] net = make_model(in_size, out_size, num_layers) opt = torch.optim.SGD(net.parameters(), lr=0.001) scaler = torch.cuda.amp.GradScaler() start_timer() for epoch in range(epochs): for input, target in zip(data, targets): with torch.autocast(device_type=device, dtype=torch.float16): output = net(input) loss = loss_fn(output, target) scaler.scale(loss).backward() scaler.step(opt) scaler.update() opt.zero_grad() end_timer_and_print("Mixed precision:")</pre>
Default precision: Total execution time = 5.277 sec	Mixed precision: Total execution time = 2.076 sec
Execution Time: 5.2 seconds	Execution Time: 2 seconds

Mixed precision training is over 2.5x faster than conventional training.

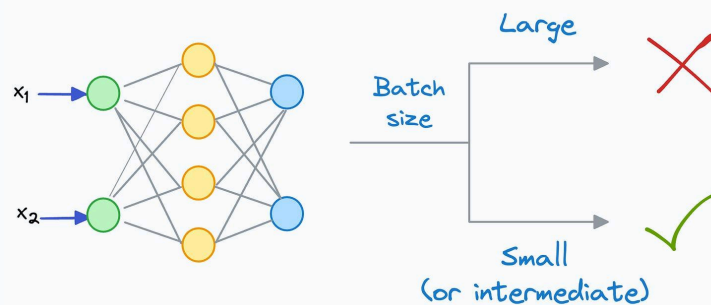
Gradient Checkpointing

Neural networks primarily utilize memory in two ways:



1. When they store model weights (this is fixed memory utilization).
2. When they are trained (this is dynamic). It happens in two ways:
 - a. During forward pass while computing and storing activations of all layers.
 - b. During backward pass while computing gradients at each layer.

The latter, i.e., dynamic memory utilization, often restricts us from training larger models with bigger batch sizes.



This is because memory utilization scales proportionately with the batch size.

That said, there's a pretty incredible technique that lets us increase the batch size while maintaining the overall memory utilization.

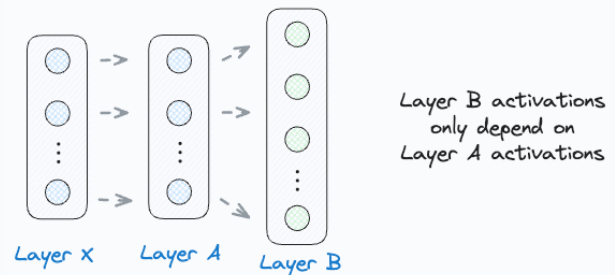
It is called Gradient checkpointing, and in my experience, it's a highly underrated technique to reduce the memory overheads of neural networks.

Let's understand this in more detail.

How gradient checkpointing works?

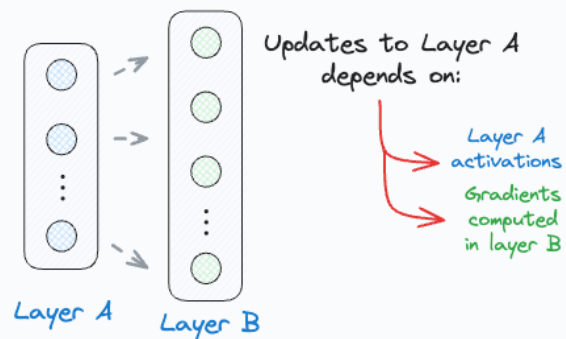
Gradient checkpointing is based on two key observations on how neural networks typically work:

1) The activations of a specific layer can be solely computed using the activations of the previous layer. For instance, in the image below, “Layer B” activations can be computed from “Layer A” activations only.



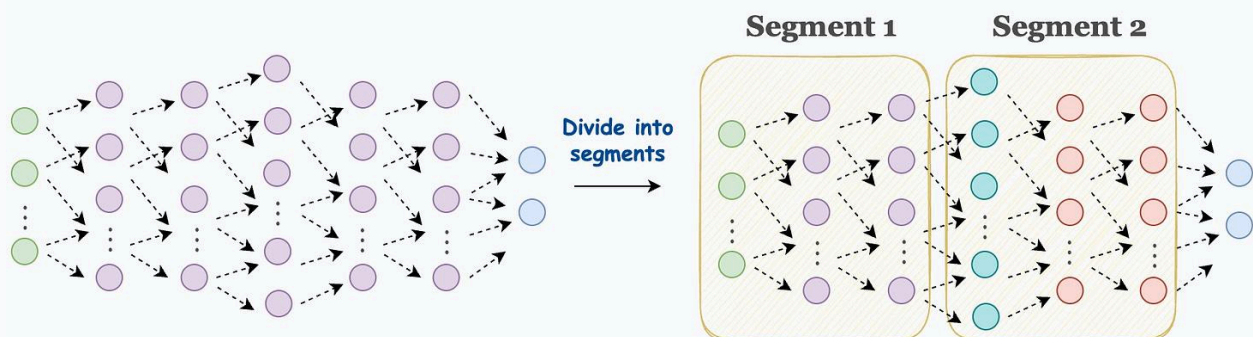
2) Updating the weights of a layer only depends on two things:

- The activations of that layer.
- The gradients computed in the next (right) layer (or rather, the running gradients).

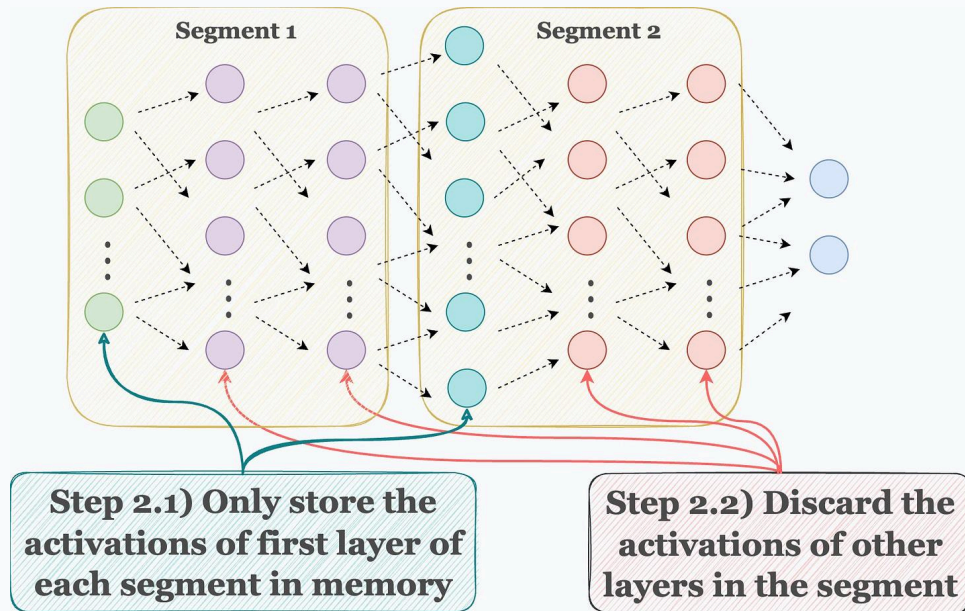


Gradient checkpointing exploits these two observations to optimize memory utilization. Here's how it works:

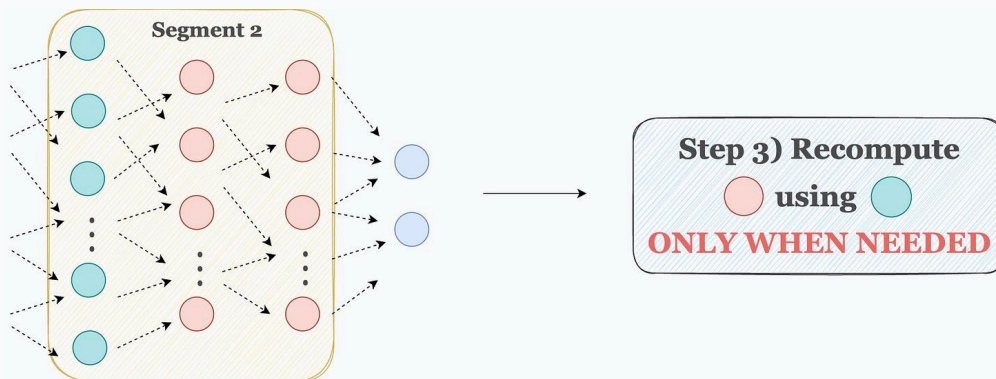
- Step 1) Divide the network into segments before the forward pass:



- Step 2) During the forward pass, only store the activations of the first layer in each segment. Discard the rest when they have been used to compute the activations of the next layer.



- Step 3) Now comes backpropagation. To update the weights of a layer, we need its activations. Thus, we recompute those activations using the first layer in that segment. For instance, as shown in the image below, to update the weights of the red layers, we recompute their activations using the activations of the cyan layer, which are already available in memory.

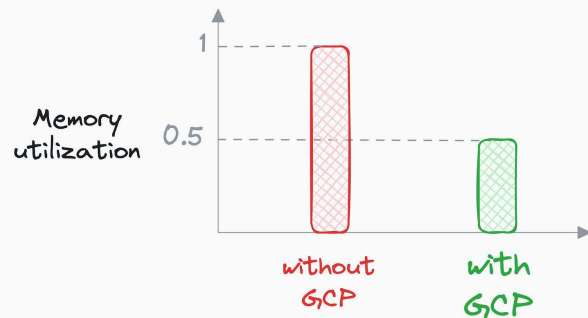


Done!

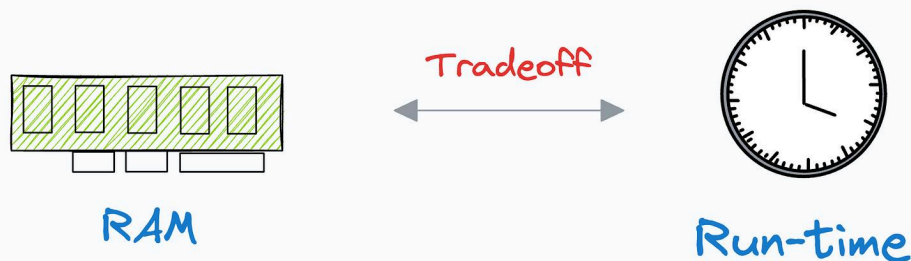
This is how gradient checkpointing works.

To summarize, the idea is that we don't need to store all the intermediate activations in memory. Instead, storing a few of them and recomputing the rest only when they are needed can significantly reduce the memory requirement. The whole idea makes intuitive sense as well.

This also allows us to train the network on larger batches of data. Typically, my observation has been that gradient checkpointing (GCP) can reduce memory usage by at least 50-60%, which is massive.



Of course, as we compute some activations twice, this does come at the cost of increased run-time, which can typically range between 15-25%. So there's always a tradeoff between memory and run-time.



That said, another advantage is that it allows us to use a larger batch size, which can slightly (not entirely though) counter the increased run-time.

Nonetheless, gradient checkpointing is an extremely powerful technique to train larger models, which I have found to be pretty helpful at times, without resorting to more intensive techniques like distributed training, for instance.

Thankfully, gradient checkpointing is also implemented by many open-source deep learning frameworks like Pytorch, etc.

Here's a demo.

Gradient checkpointing in PyTorch

To utilize this, we begin by importing the necessary libraries and functions:

```
import torch
import torch.nn as nn
from torch.utils.checkpoint import checkpoint_sequential
```

Next, we define our neural network:

```
class SimpleNet(nn.Module):
    def __init__(self):
        super(SimpleNet, self).__init__()
        self.layer1 = nn.Linear(1000, 500)
        self.layer2 = nn.Linear(500, 200)
        self.layer3 = nn.Linear(200, 100)
        self.layer4 = nn.Linear(100, 10)

    def forward(self, x):
        # Define layers
        layers = torch.nn.Sequential(
            self.layer1, nn.ReLU(),
            self.layer2, nn.ReLU(),
            self.layer3, nn.ReLU(),
            self.layer4)

        # Checkpoint intermediate activations
        x = checkpoint_sequential(layers,
                                segments=2,
                                input=x)

        return x
```

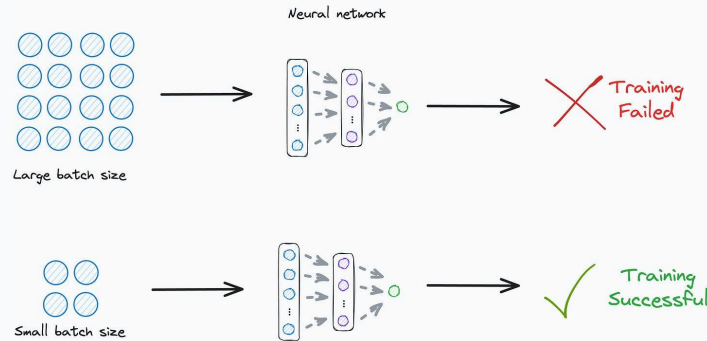
Gradient
Checkpointing

As demonstrated above, in the forward method, we use the `checkpoint_sequential` method to use gradient checkpointing and divide the network into two segments.

Next, we can proceed with network training as we usually would.

Gradient Accumulation

Under memory constraints, it is always recommended to train the neural network with a small batch size.



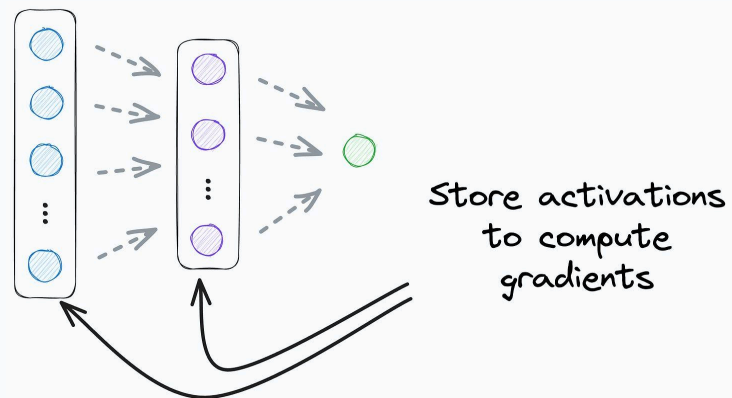
Despite that, there's a technique called **gradient accumulation**, which lets us (logically) increase batch size without explicitly increasing the batch size.

Confused?

Let's understand in this chapter. But before that, we must understand...

Why do neural networks typically explode during training?

The primary memory overhead in a neural network comes from backpropagation. This is because, during backpropagation, we must store the layer activations in memory. After all, they are used to compute the gradients.



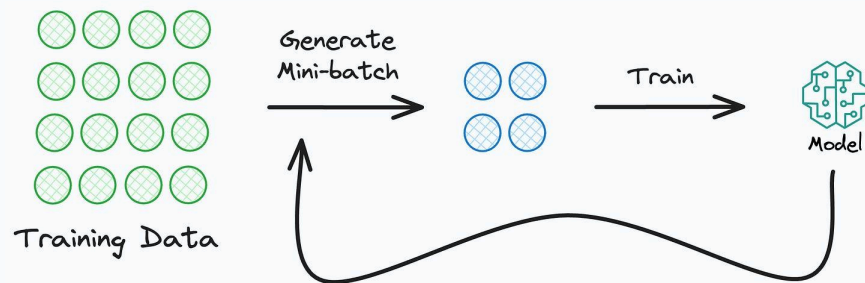
The bigger the network, the more activations a network must store in memory. Also, under memory constraints, having a large batch size will result in:

- storing many activations
- using those many activations to compute the gradients

This may lead to more resource consumption than available — resulting in training failure. But by reducing the batch size, we can limit the memory usage and train the network.

What is Gradient Accumulation and how does it help in increasing batch size in memory constraints?

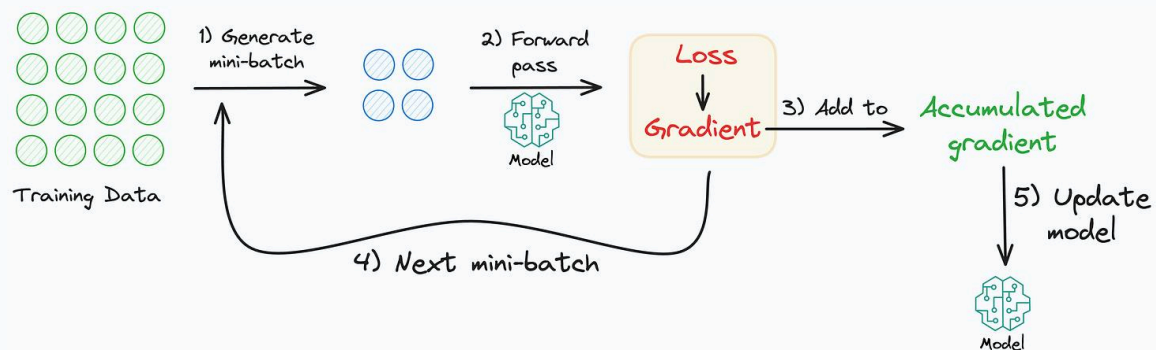
Consider we are training a neural network on mini-batches.



We train the network as follows:

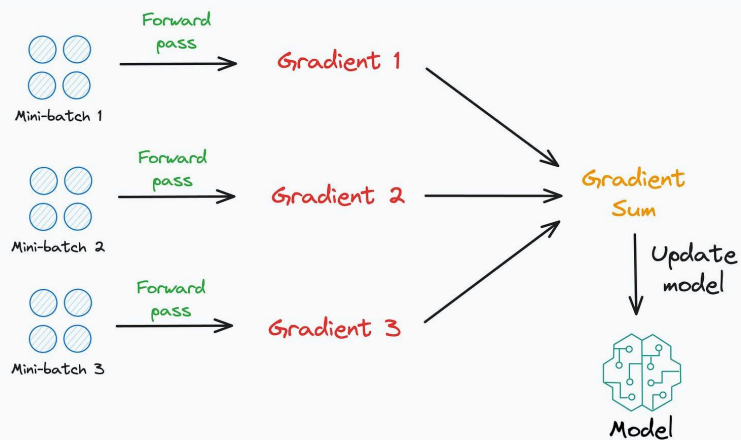
- On every mini-batch:
 - Run the forward pass while storing the activations.
 - During backward pass:
 - Compute the loss
 - Compute the gradients
 - Update the weights

Gradient accumulation modifies the last step of the backward pass, i.e., weight updates. More specifically, instead of updating the weights on every mini-batch, we can do this:



1. Run the forward pass on a mini-batch.
2. Compute the gradient values for weights in the network.
3. Don't update the weights yet.
4. Run the forward pass on the next mini-batch.
5. Compute the gradient values for weights and add them to the gradients obtained in step 2.
6. Repeat steps 3-5 for a few more mini-batches.
7. Update the weights only after processing a few mini-batches.

This technique works because accumulating the gradients across multiple mini-batches results in the same sum of gradients as if we were processing them together. Thus, logically speaking, using gradient accumulation, we can mimic a larger batch size without having to explicitly increase the batch size.



For instance, say we want to use a batch size of 64. However, current memory can only support a batch size of 16.

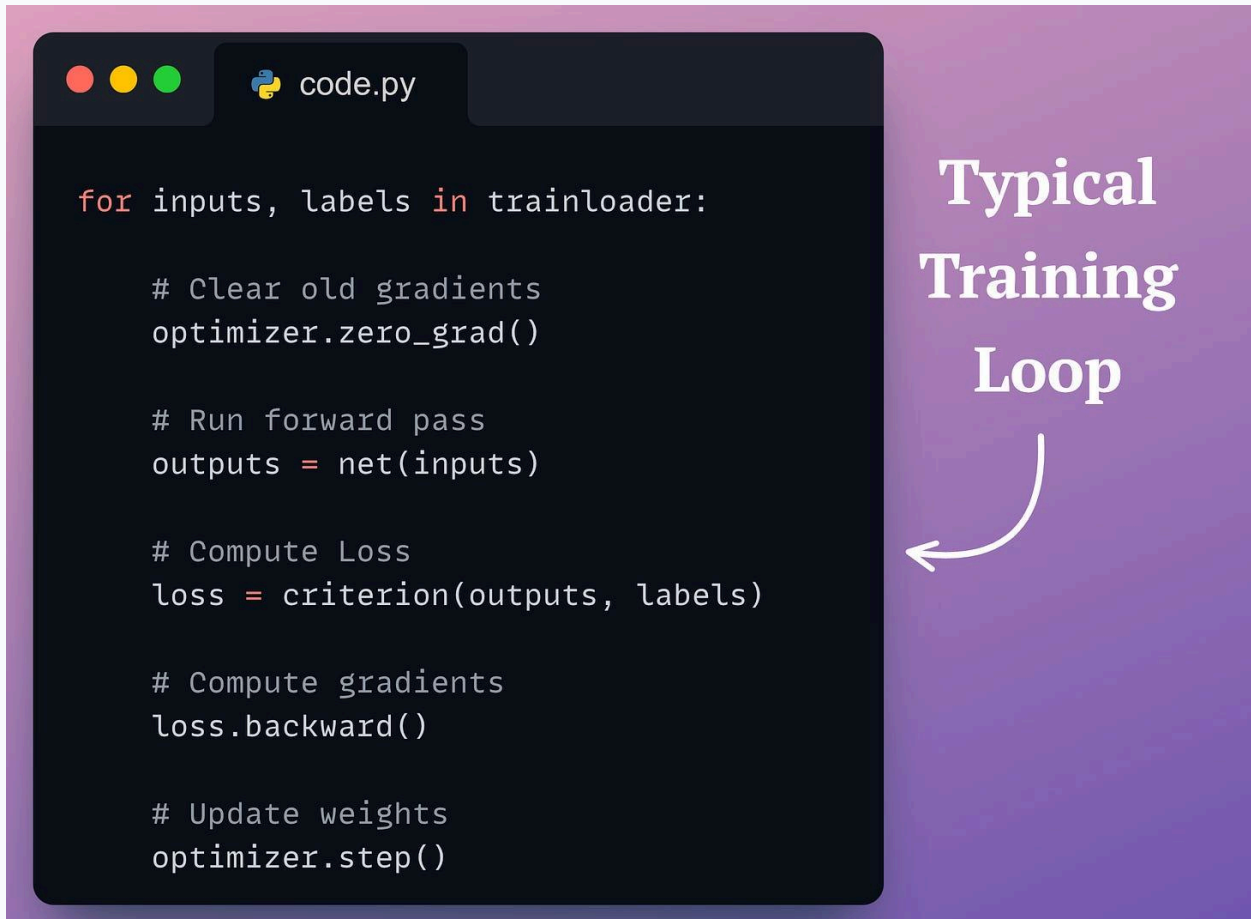
No worries!

- We can use a batch size of size 16.
- We can accumulate the gradients from every mini-batch.
- We can update the weights only once every 8 mini-batches.

Thus, effectively, we used a batch size of $16 \times 8 (=128)$ instead of what we originally intended — 64.

Implementation

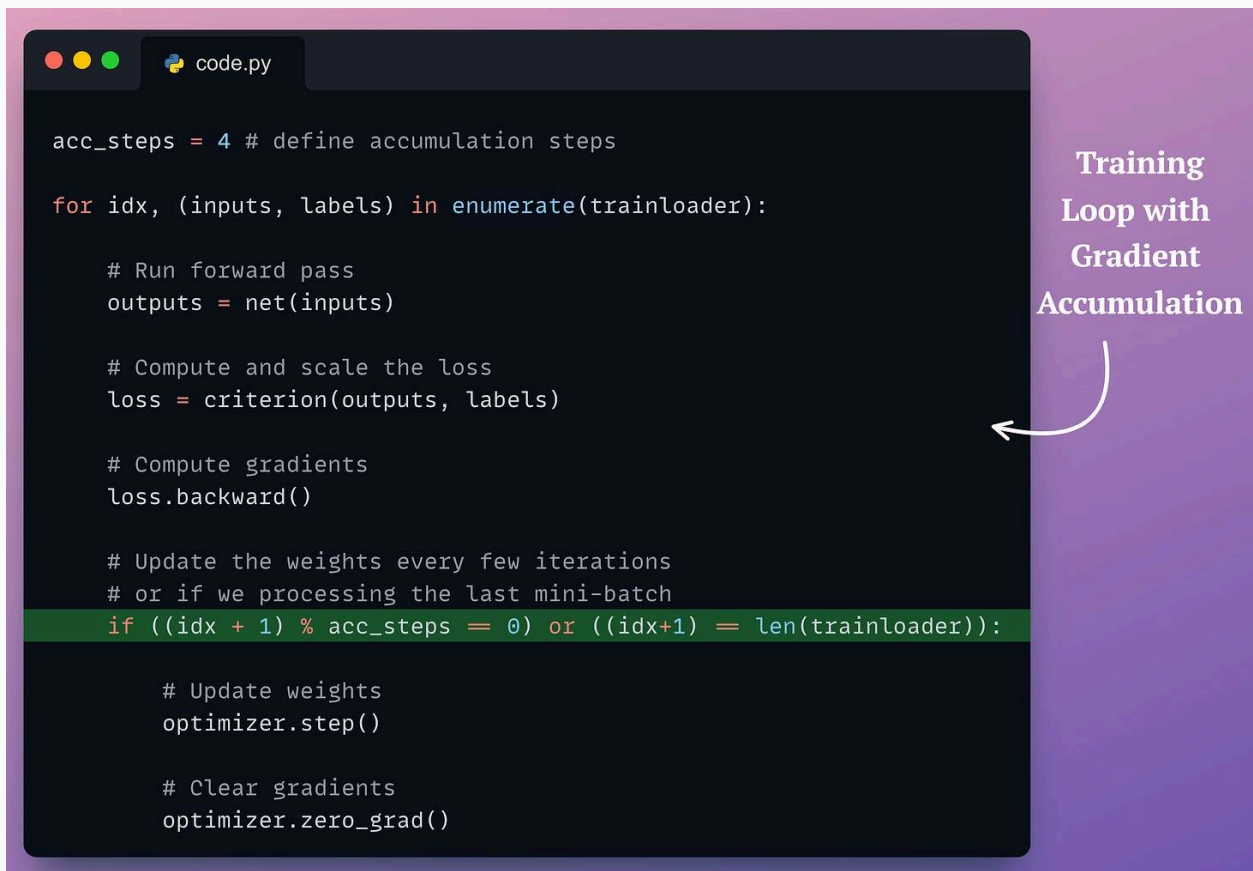
Let's look at how we can implement this. In PyTorch, a typical training loop is implemented as follows:



- We clear the gradients
- Run the forward pass
- Compute the loss
- Compute the gradients
- Update the weights

However, as discussed earlier, if needed, we can only update the weights after a few iterations. Thus, we must continue to accumulate the gradients, which is precisely what `loss.backward()` does.

Also, as `optimizer.zero_grad()` clears the gradients, we must only execute it after updating the weights. This idea is implemented below:

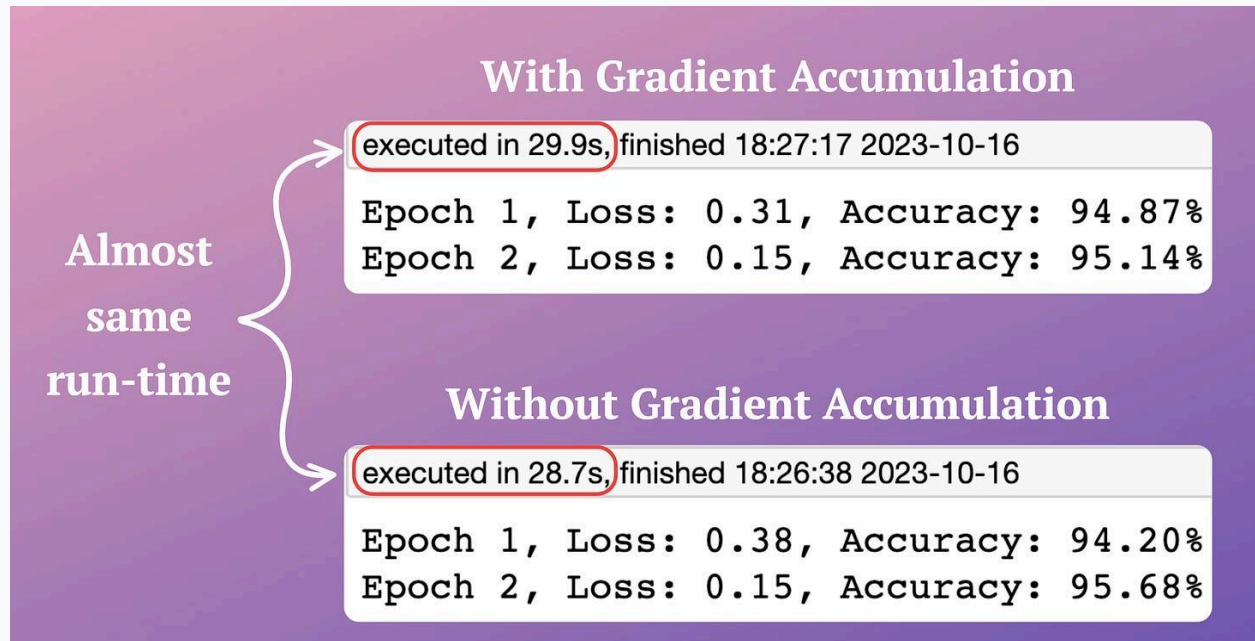


- First, we define `acc_steps` — the number of mini-batches after which we want to update the weights.
- Next, we run the forward pass.
- Moving on, we compute the loss and the gradients.
- As discussed earlier, we will not update the weights yet and instead let the gradients accumulate for a few more mini-batches.
- We only update the weights when the if condition is true.
- After updating, we clear the accumulated gradients.

This way, we can optimize neural network training in memory-constrained settings.

Departing note

Before we end, it is essential to note that gradient accumulation is NOT a remedy to improve run-time in memory-constrained situations. In fact, we can also verify this from my experiment:



Instead, its objective is to reduce overall memory usage.

Of course, it's true that we are updating the weights only after a few iterations. So, it will be a bit faster than updating on every iteration. Yet, we are still processing and computing gradients on small mini-batches, which is the core operation here.

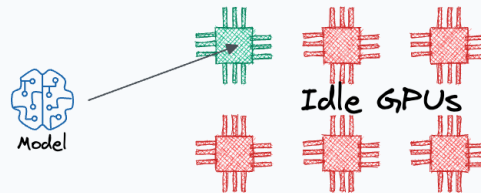
Nonetheless, the good thing is that even if you are not under memory constraints, you can still use gradient accumulation.

- Specify your typical batch size.
- Run forward pass.
- Compute loss and gradients.
- Update only after a few iterations.

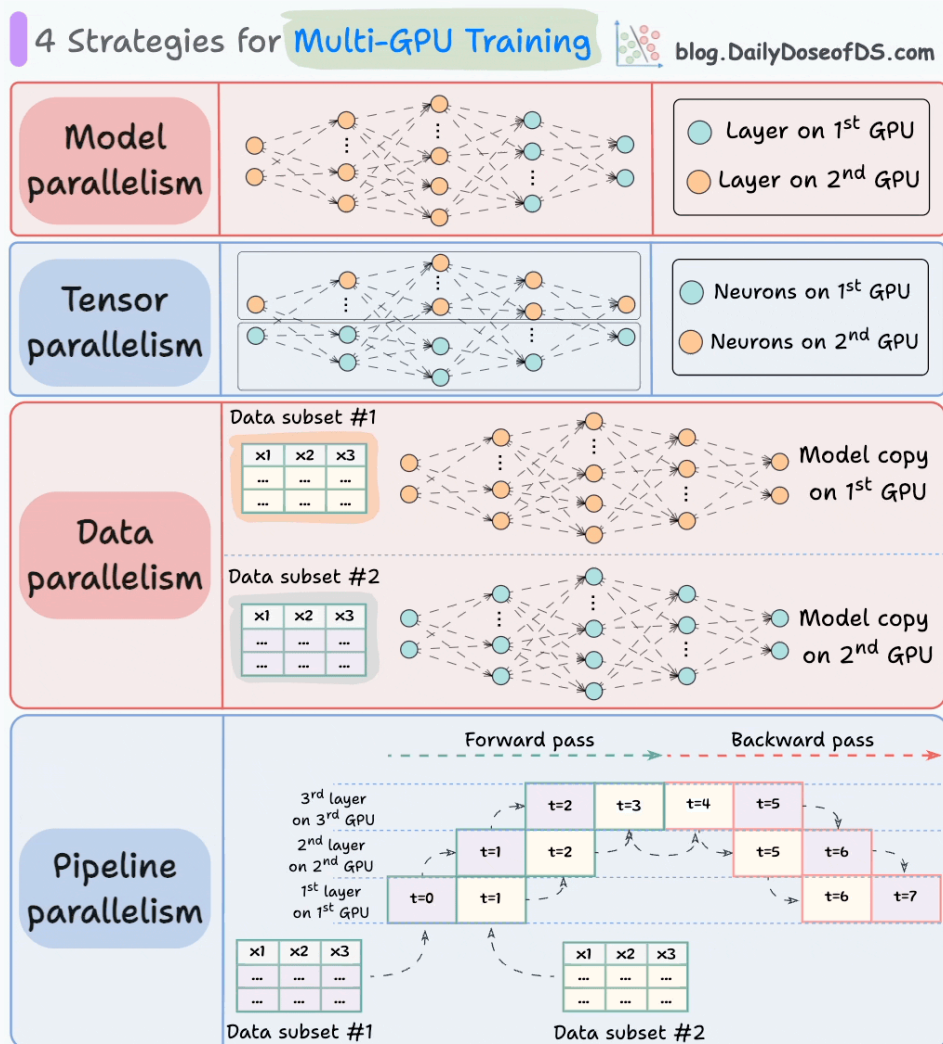
You can download the notebook here: <https://bit.ly/3xNCfFt>.

4 Strategies for Multi-GPU Training

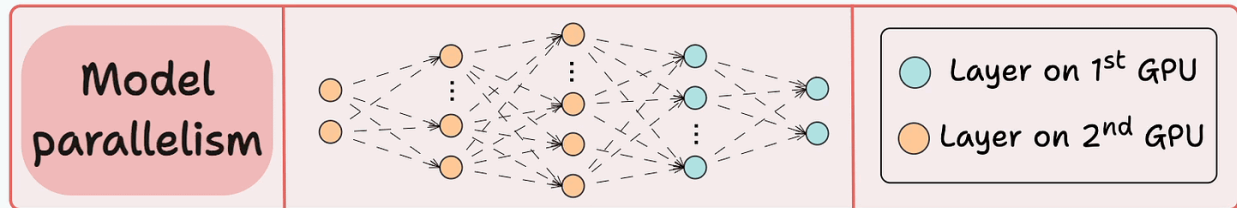
By default, deep learning models only utilize a single GPU for training, even if multiple GPUs are available.



An ideal way to proceed (especially in big-data settings) is to distribute the training workload across multiple GPUs. The graphic below depicts four common strategies for multi-GPU training:

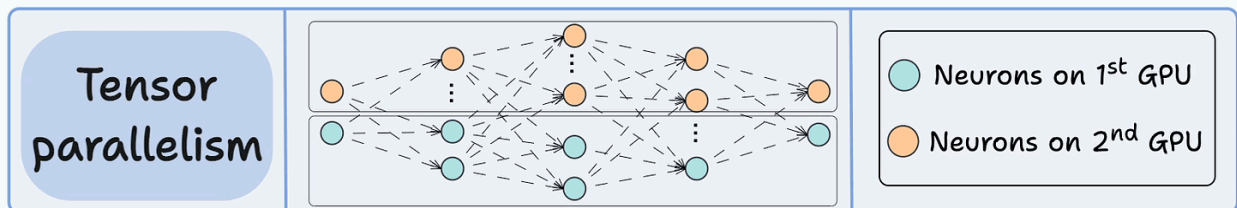


#1) Model parallelism

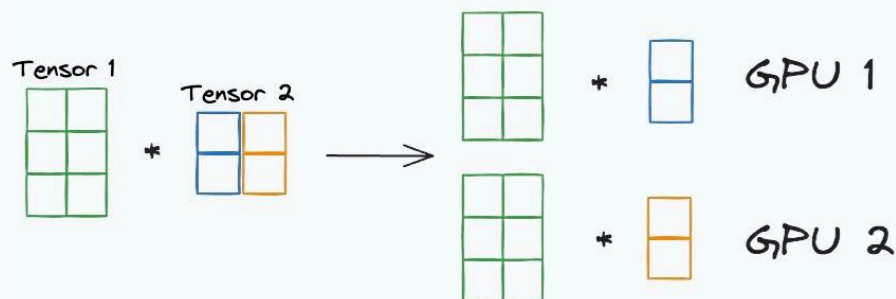


- Different parts (or layers) of the model are placed on different GPUs.
- Useful for huge models that do not fit on a single GPU.
- However, model parallelism also introduces severe bottlenecks as it requires data flow between GPUs when activations from one GPU are transferred to another GPU.

#2) Tensor parallelism

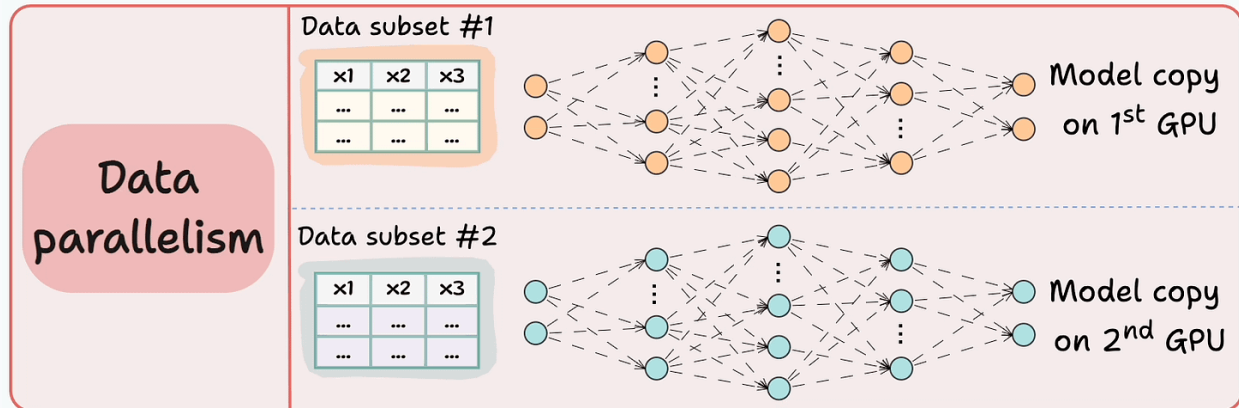


- Distributes and processes individual tensor operations across multiple devices or processors.
- It is based on the idea that a large tensor operation, such as matrix multiplication, can be divided into smaller tensor operations, and each smaller operation can be executed on a separate device or processor.



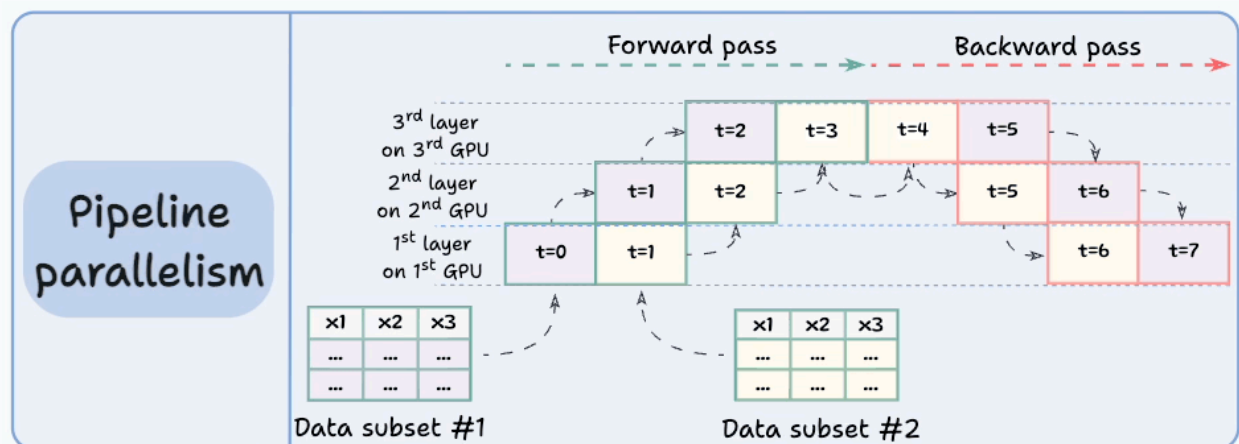
- Such parallelization strategies are inherently built into standard implementations of PyTorch and other deep learning frameworks, but they become much more pronounced in a distributed setting.

#3) Data parallelism



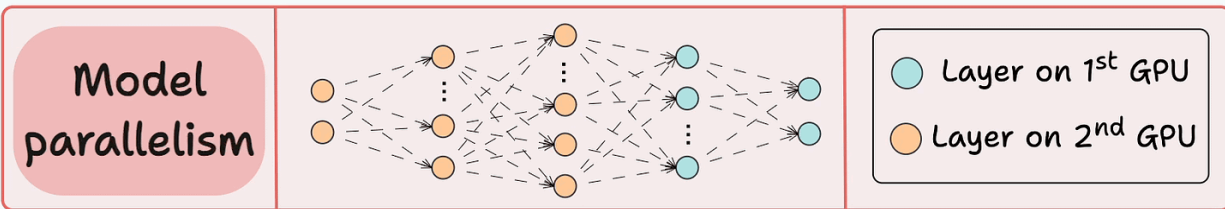
- Replicate the model across all GPUs.
- Divide the available data into smaller batches, and each batch is processed by a separate GPU.
- The updates (or gradients) from each GPU are then aggregated and used to update the model parameters on every GPU.

#4) Pipeline parallelism

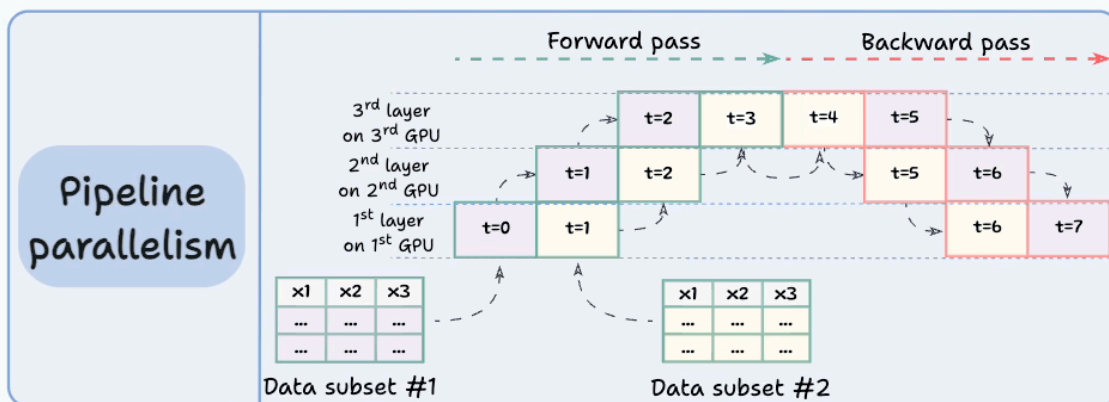


- This is often considered a combination of data parallelism and model parallelism.

- So the issue with standard model parallelism is that 1st GPU remains idle when data is being propagated through layers available in 2nd GPU:



- Pipeline parallelism addresses this by loading the next micro-batch of data once the 1st GPU has finished the computations on the 1st micro-batch and transferred activations to layers available in the 2nd GPU. The process looks like this:
 - 1st micro-batch passes through the layers on 1st GPU.
 - 2nd GPU receives activations on 1st micro-batch from 1st GPU.
 - While the 2nd GPU passes the data through the layers, another micro-batch is loaded on the 1st GPU.
 - And the process continues.
- GPU utilization drastically improves this way. This is evident from the animation below where multi-GPUs are being utilized at the same timestamp (look at $t=1$, $t=2$, $t=5$, and $t=6$):

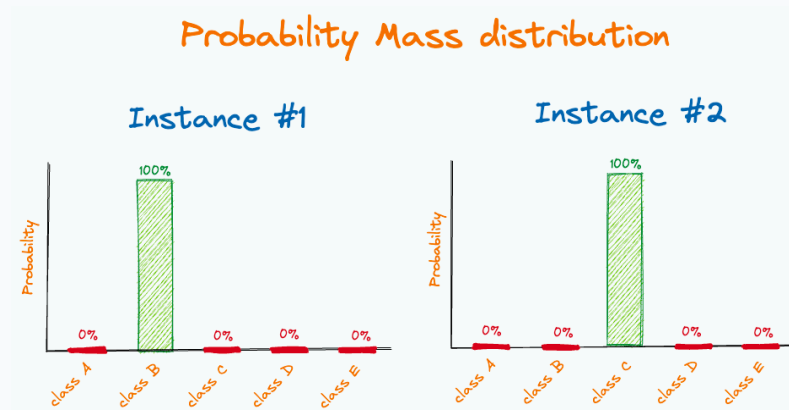


Those were four common strategies for multi-GPU training.

Miscellaneous

Label Smoothing

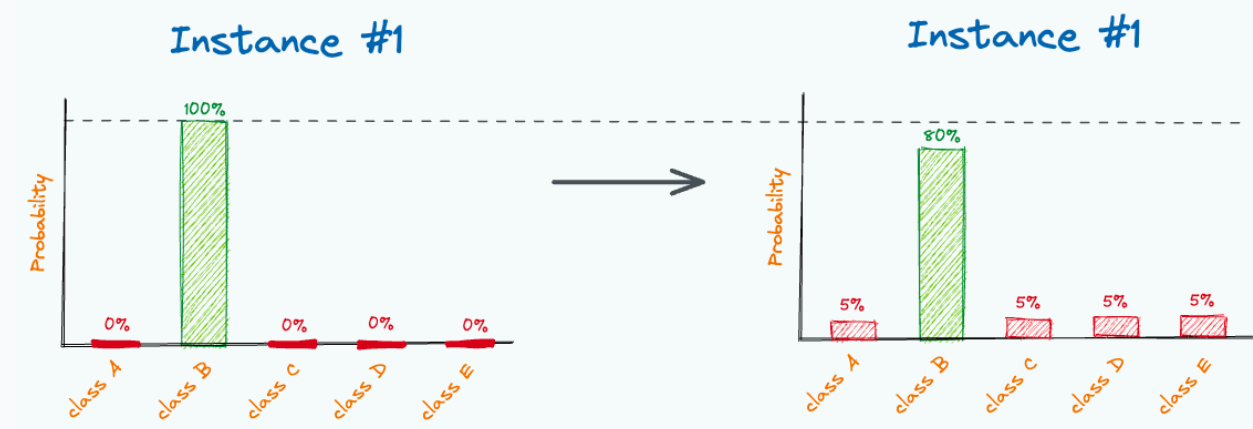
For every instance in single-label classification datasets, the entire probability mass belongs to a single class, and the rest are zero. This is depicted below:



The issue is that, at times, such label distributions excessively motivate the model to learn the true class for every sample with pretty high confidence. This can impact its generalization capabilities.

Label smoothing is a lesser-talked regularisation technique that elegantly addresses this issue.

Label Smoothing

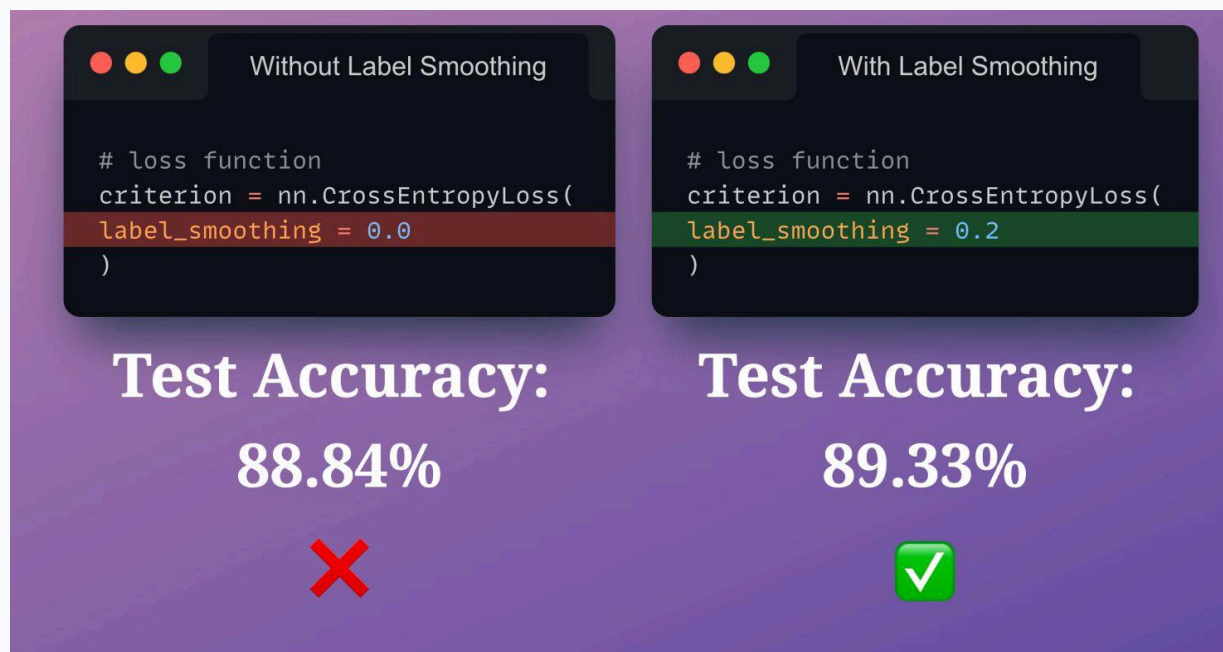


As depicted above, with label smoothing:

- We intentionally reduce the probability mass of the true class slightly.
- The reduced probability mass is uniformly distributed to all other classes.

Simply put, this can be thought of as asking the model to be “less overconfident” during training and prediction while still attempting to make accurate predictions.

The efficacy of this technique is evident from the image below:



In this experiment, I trained two neural networks on the Fashion MNIST dataset with the exact same weight initialization.

- One without label smoothing.
- Another with label smoothing.

The model with label smoothing resulted in a better test accuracy, i.e., better generalization.

When not to use label smoothing?

After using label smoothing for many of my projects, I have also realized that it is not well suited for all use cases. So it's important to know when you should not use it.

See, if you only care about getting the final prediction correct and improving generalization, label smoothing will be a pretty handy technique. However, I wouldn't recommend utilizing it if you care about:

- Getting the prediction correct.
- **And** understanding the model's confidence in generating a prediction.

This is because as we discussed above, label smoothing guides the model to become “less overconfident” about its predictions. Thus, we typically notice a drop in the confidence values for every prediction, as depicted below:



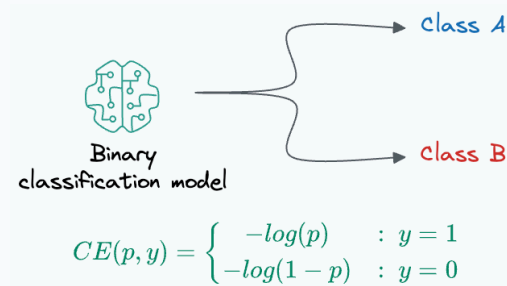
On a specific test instance:

- The model without label smoothing outputs 99% probability for class 3.
- With label smoothing, although the prediction is still correct, the confidence drops to 74%.

This is something to keep in mind when using label smoothing. Nonetheless, the technique is indeed pretty promising for regularizing deep learning models. You can download the code notebook for this chapter here: <https://bit.ly/4ePt08d>.

Focal loss

Binary classification tasks are typically trained using the binary cross entropy (BCE) loss function:



A diagram showing a 'Binary classification model' represented by a brain icon. Two arrows branch out from the model to 'class A' and 'class B'.

$$CE(p, y) = \begin{cases} -\log(p) & : y = 1 \\ -\log(1 - p) & : y = 0 \end{cases}$$

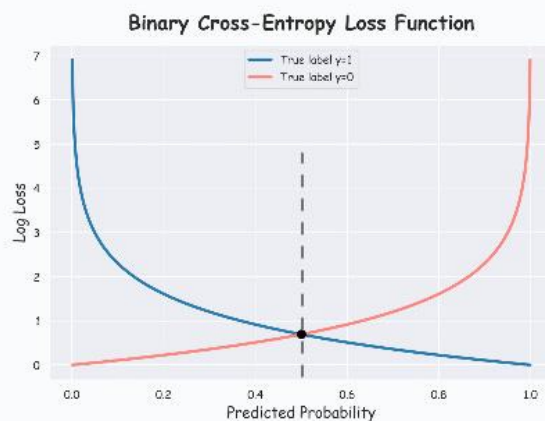
For notational convenience, if we define p_t as the following:

$$p_t = \begin{cases} p & : y = 1 \\ 1 - p & : y = 0 \end{cases}$$

...then we can also write the cross-entropy loss function as:

$$CE(p, y) = -\log(p_t)$$

That said, one limitation of BCE loss is that it weighs probability predictions for both classes equally, which is evident from its symmetry:



Symmetric
about
 $p_t = 0.5$

For more clarity, consider the table below, which depicts two instances, one from the minority class and another from the majority class, both with the same loss:

True label	$y = 1$ minority class	$y = 0$ majority class
Probability output	$\hat{y} = 0.3$	$\hat{y} = 0.7$
log loss	$-\log(0.3)$	$-\log(0.3)$

This causes problems when we use BCE for imbalanced datasets, wherein most instances from the dominating class are “easily classifiable.” Thus, a loss value of, say, $-\log(0.3)$ from the majority class instance should (ideally) be weighed LESS than the same loss value from the minority class.

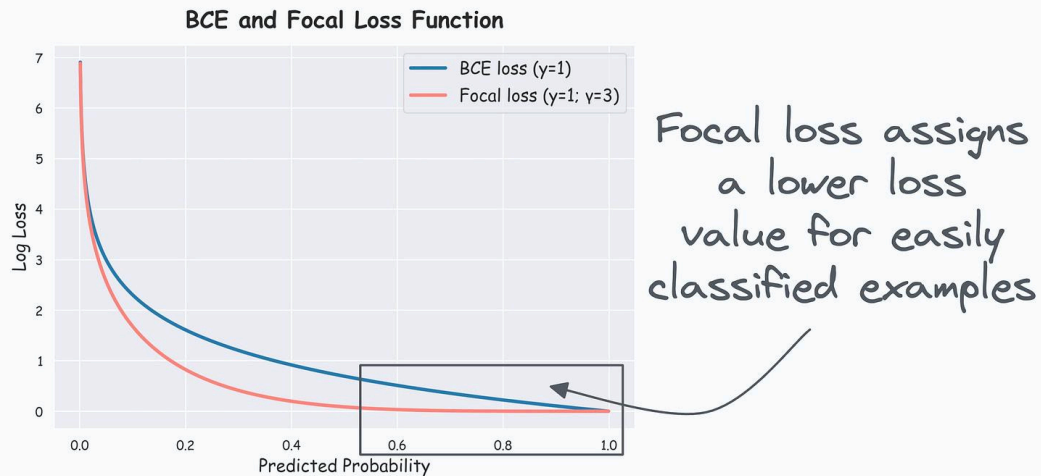
$$\begin{array}{ccc}
 -\log(0.3) & >> & -\log(0.3) \\
 \text{from minority} & & \text{from majority} \\
 \text{class} & & \text{class}
 \end{array}$$

Focal loss is a pretty handy and useful alternative to address this issue. It is defined as follows:

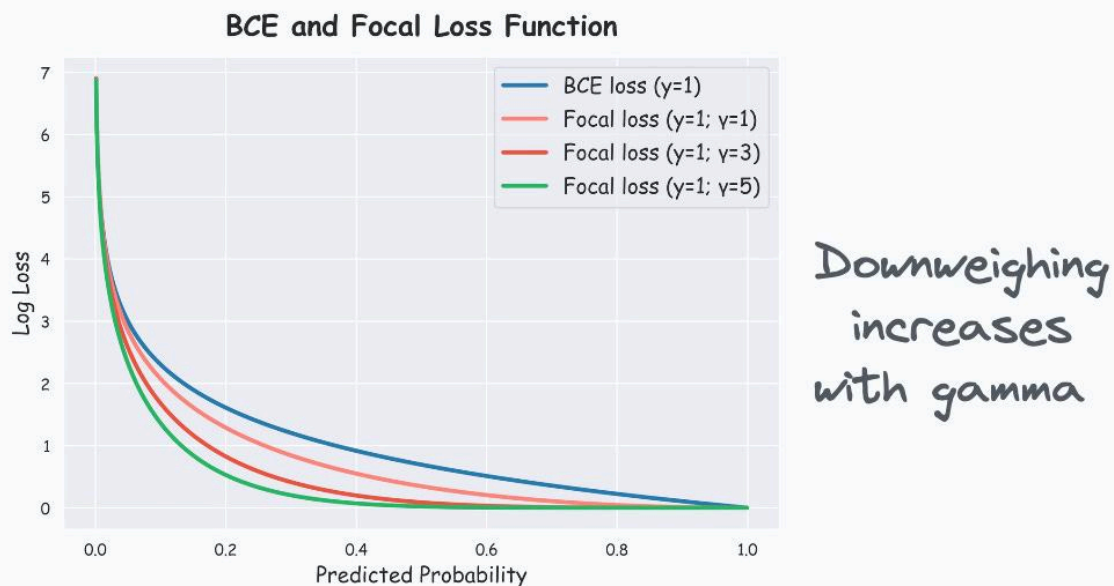
$$\begin{aligned}
 CE(p, y) &= -\log(p_t) \\
 Focal(p, y) &= -(1 - p_t)^\gamma \cdot \log(p_t)
 \end{aligned}$$

As depicted above, it introduces an additional multiplicative factor called downweighing, and the parameter γ (**Gamma**) is a hyperparameter.

Plotting BCE (class $y=1$) and Focal loss (for class $y=1$ and $\gamma=3$), we get the following curve:

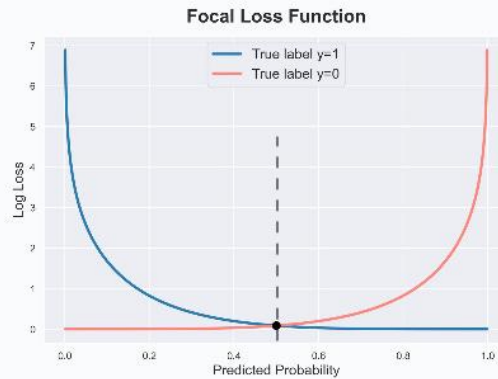


As shown in the figure above, focal loss reduces the contribution of the predictions the model is pretty confident about. Also, the higher the value of γ (Gamma), the more downweighing takes place, as shown in this plot below:



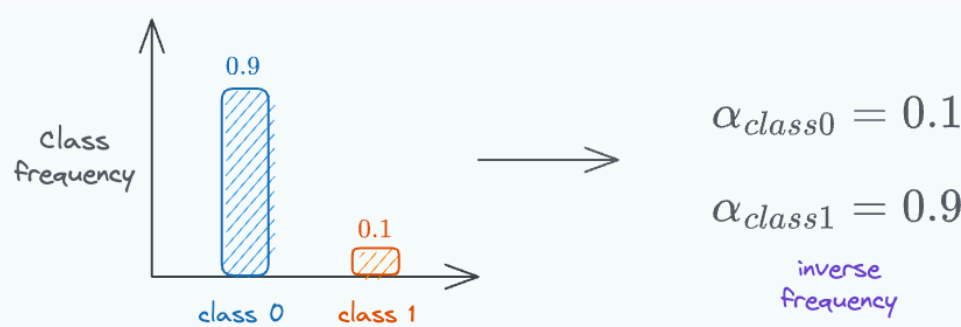
Moving on, while the Focal loss function reduces the contribution of confident predictions, we aren't done yet.

The focal loss function now is still symmetric like BCE:



Symmetric
about
 $p_t = 0.5$

To address this, we must add another weighing parameter (α), which is the inverse of the class frequency, as depicted below:

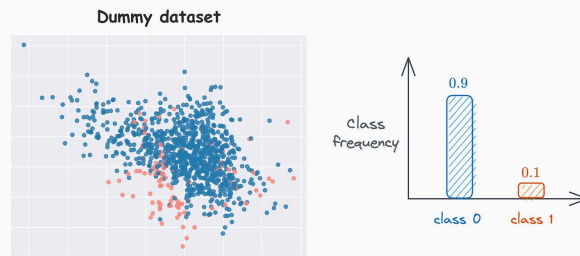


The α parameter is the inverse of the class frequency. Thus, the final loss function comes out to be the following:

$$Focal(p, y) = - \underbrace{\alpha_t}_{\text{class inverse weighing}} \cdot \underbrace{(1 - p_t)^\gamma}_{\text{Probability downweighing}} \cdot \log(p_t)$$

By using both downweighing and inverse weighing, the model gradually learns patterns specific to the hard examples instead of always being overly confident in predicting easy instances.

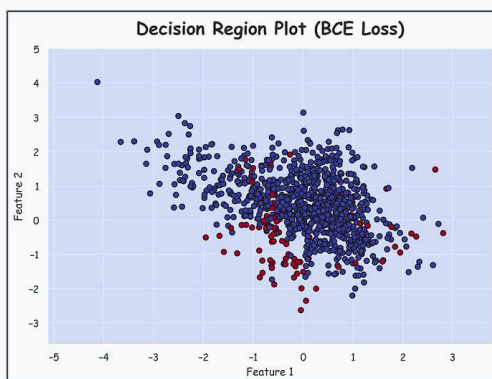
To test the efficacy of focal loss in a class imbalance setting, I created a dummy classification dataset with a 90:10 imbalance ratio:



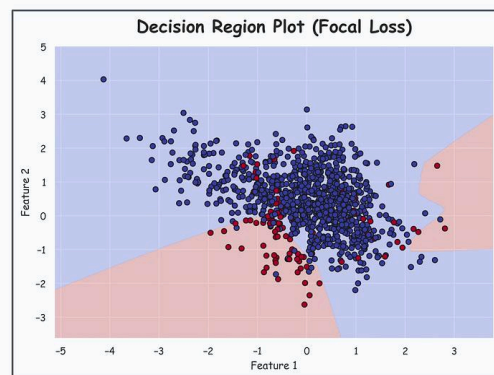
Next, I trained two neural network models (with the same architecture of 2 hidden layers):

- One with BCE loss
- Another with Focal loss

The decision region plot and test accuracy for these two models is depicted below:



Model always predicts majority class
(Test accuracy = 90%)



Model learns some minority class patterns
(Test accuracy = 93%)

It is clear that:

- The model trained with BCE loss (left) always predicts the majority class.
- The model trained with focal loss (right) focuses relatively more on minority class patterns. As a result, it performs better.

Download this Jupyter notebook to get started with Focal loss:

<https://bit.ly/45XzNZC>.

How Dropout Actually Works?

Some time back, I was invited by a tech startup to conduct their ML interviews. I interviewed 12 candidates and mostly asked practical ML questions.

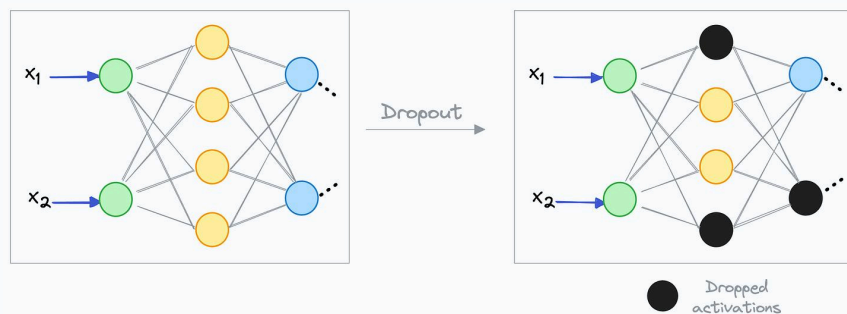
However, there were some conceptual questions as well, like the one below, which I intentionally asked every candidate:

How does Dropout work?

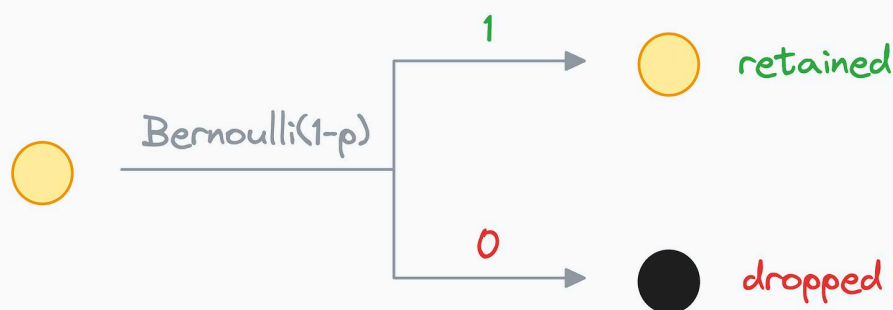
Pretty simple, right? Apparently, every candidate gave me an incomplete answer, which I have mentioned below:

Candidates' Answer

In a gist, the idea is to zero out neurons randomly in a neural network. This is done to regularize the network.



Dropout is only applied during training, and which neuron activations to zero out (or drop) is decided using a Bernoulli distribution:



“p” is the dropout probability specified in, say, PyTorch $\rightarrow$ `nn.Dropout(p)`.

My follow-up question: Is there anything else that we do in Dropout?

Candidates: No, that's it. We only zero out neurons and train the network as we usually would.

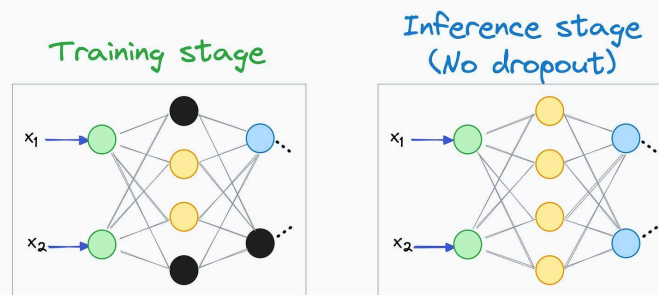
Now, coming back to the topic...

Of course, I am not saying that the above details are incorrect. They are correct.

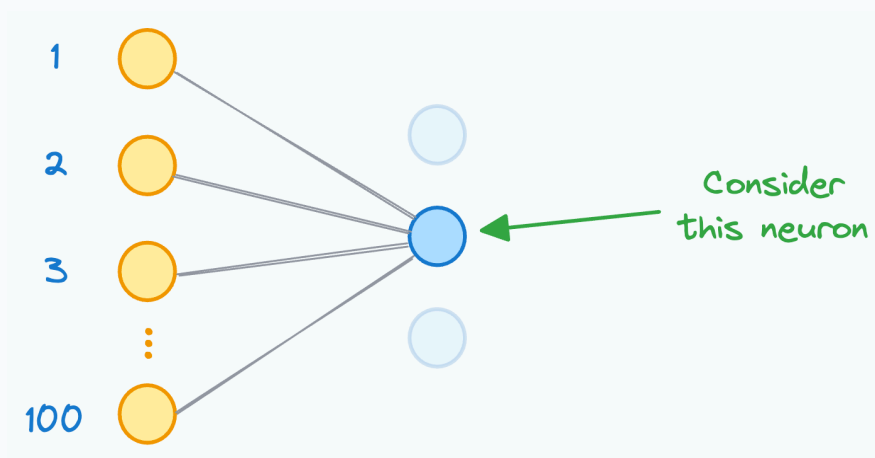
However, this is just 50% of how Dropout works, and disappointingly, most resources don't cover the remaining 50%. If you too are only aware of the 50% details I mentioned above, continue reading.

How Dropout actually works?

To begin, we must note that Dropout is only applied during training, but not during the inference/evaluation stage:

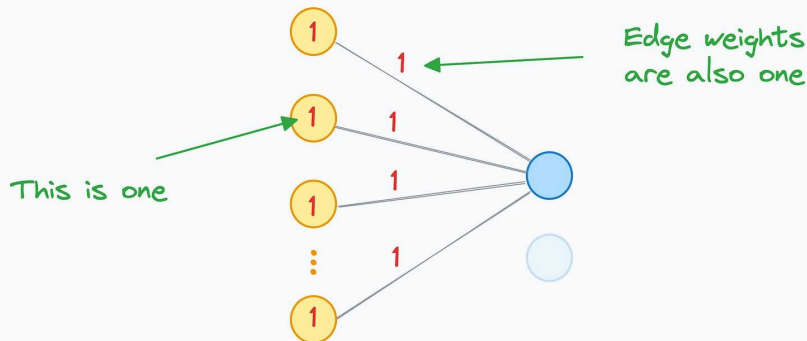


Now, consider that a neuron's input is computed using 100 neurons in the previous hidden layer:

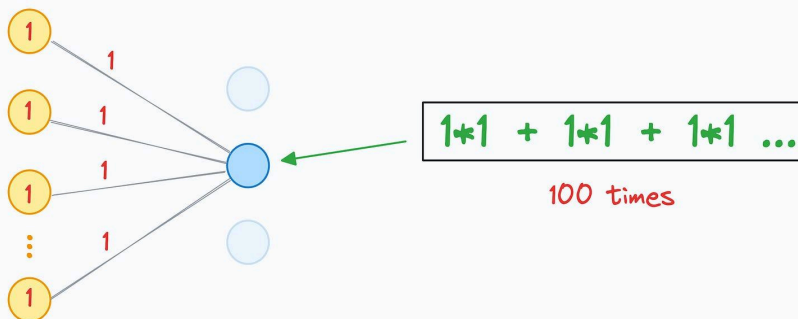


For simplicity, let's assume a couple of things here:

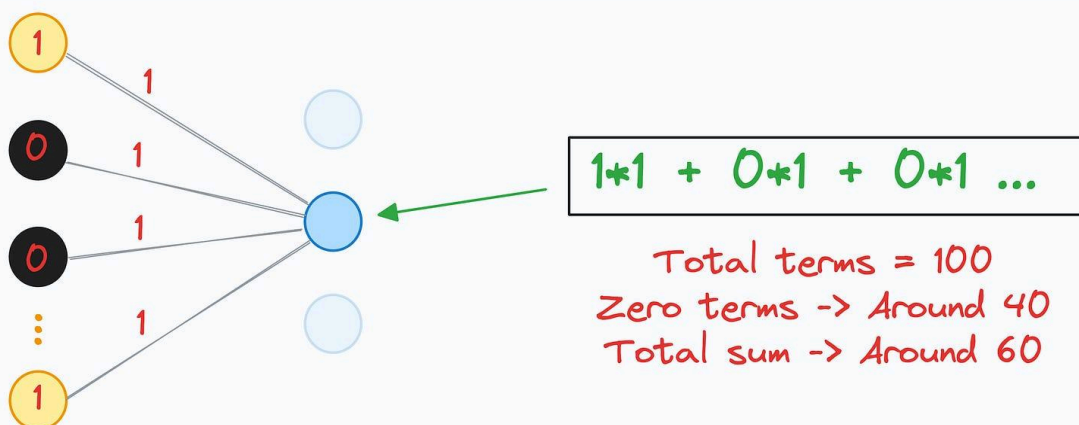
- The activation of every yellow neuron is 1.
- The edge weight from the yellow neurons to the blue neuron is also 1.



As a result, the input received by the blue neuron will be 100, as depicted below:



Now, during training, if we were using Dropout with, say, a 40% dropout rate, then roughly 40% of the yellow neuron activations would have been zeroed out. As a result, the input received by the blue neuron would have been around 60:



However, the above point is only valid for the training stage.

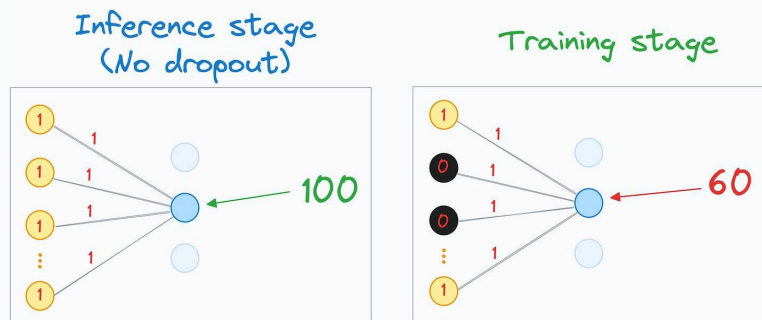
If the same scenario had existed during the inference stage instead, then the input received by the blue neuron would have been 100.

Thus, under similar conditions:

- The input received during training → 60.
- The input received during inference → 100.

Do you see any problem here?

During training, the average neuron inputs are significantly lower than those received during inference.



More formally, using Dropout significantly affects the scale of the activations. However, it is desired that the neurons throughout the model must receive the roughly same mean (or expected value) of activations during training and inference. To address this, Dropout performs one additional step.

This idea is to scale the remaining active inputs during training. The simplest way to do this is by scaling all activations during training by a factor of $1/(1-p)$, where p is the dropout rate. For instance, using this technique on the neuron input of 60, we get the following (recall that we set $p=40\%$):

$$\begin{array}{c} 60 \\ \text{blue circle} \end{array} \longrightarrow 60 * \frac{1}{1 - 0.4} = \boxed{100} \quad \text{output rescaled}$$

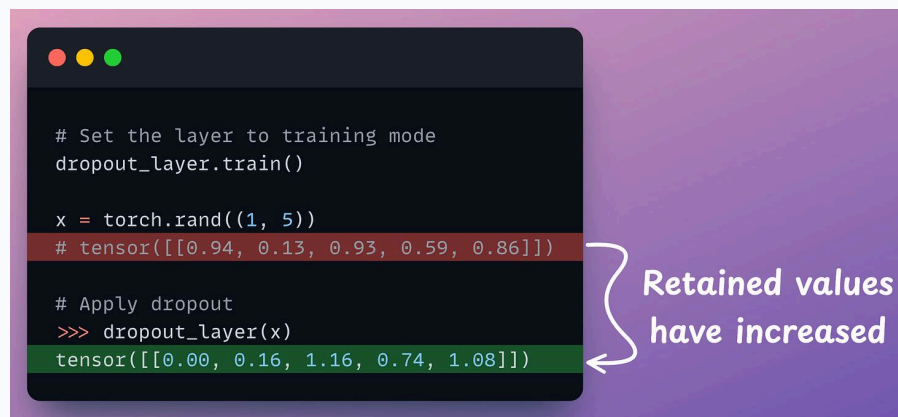
As depicted above, scaling the neuron input brings it to the desired range, which makes training and inference stages coherent for the network.

Verifying experimentally

In fact, we can verify that typical implementations of Dropout, from PyTorch, for instance, do carry out this step. Let's define a dropout layer as follows:



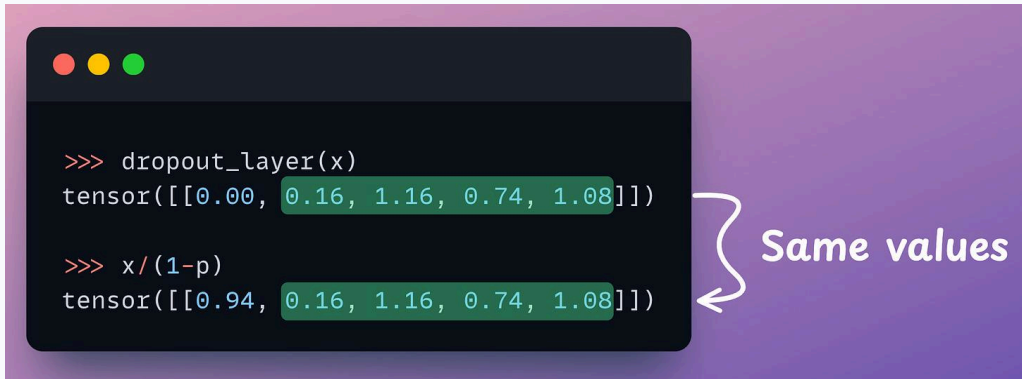
Now, let's consider a random tensor and apply this dropout layer to it:



As depicted above, the retained values have increased.

- The second value goes from 0.13 $\rightarrow$ 0.16.
- The third value goes from 0.93 $\rightarrow$ 1.16.
- and so on...

What's more, the retained values are precisely the same as we would have obtained by explicitly scaling the input tensor with $1/(1-p)$:

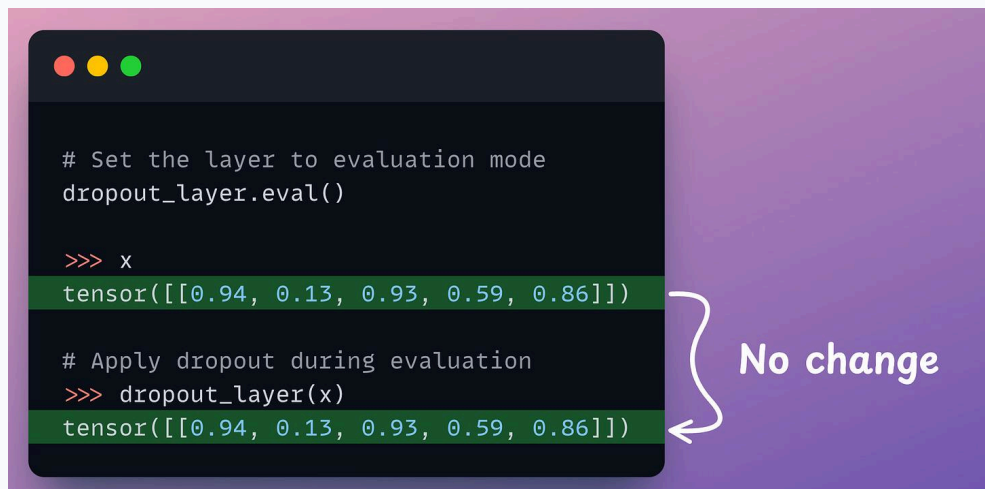


```
>>> dropout_layer(x)
tensor([[0.00, 0.16, 1.16, 0.74, 1.08]])

>>> x/(1-p)
tensor([[0.94, 0.16, 1.16, 0.74, 1.08]])
```

Same values

If we were to do the same thing in evaluation mode instead, we notice that no value is dropped and no scaling takes place either, which makes sense as Dropout is only used during training:



```
# Set the layer to evaluation mode
dropout_layer.eval()

>>> x
tensor([[0.94, 0.13, 0.93, 0.59, 0.86]])

# Apply dropout during evaluation
>>> dropout_layer(x)
tensor([[0.94, 0.13, 0.93, 0.59, 0.86]])
```

No change

This is the remaining 50% details, which, in my experience, most resources do not cover, and as a result, most people aren't aware of.

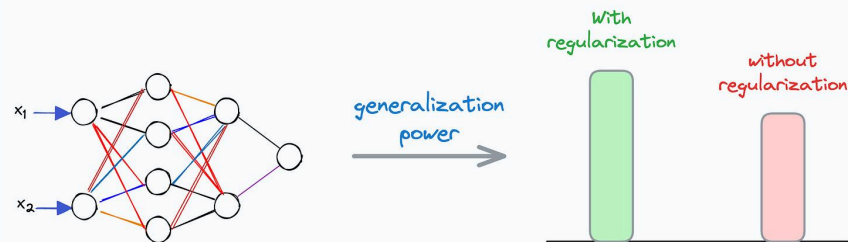
But it is a highly important step in Dropout, which maintains numerical coherence between training and inference stages.

With that, now you know 100% of how Dropout works.

Next, let's discuss an issue with Dropout in case of CNNs.

Issues with Dropout in CNNs

When it comes to training neural networks, it is always recommended to use Dropout to improve its generalization power.

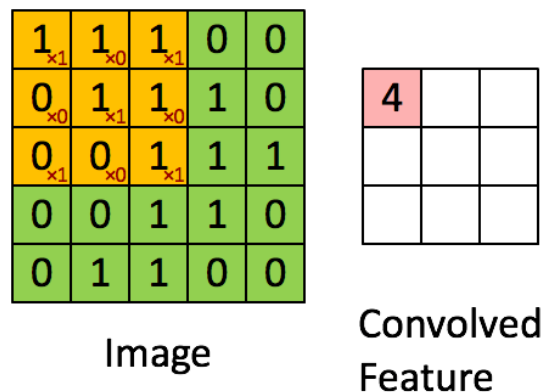


This applies not just to CNNs but to all other neural networks. And I am sure you already know the above details, so let's get into the interesting part.

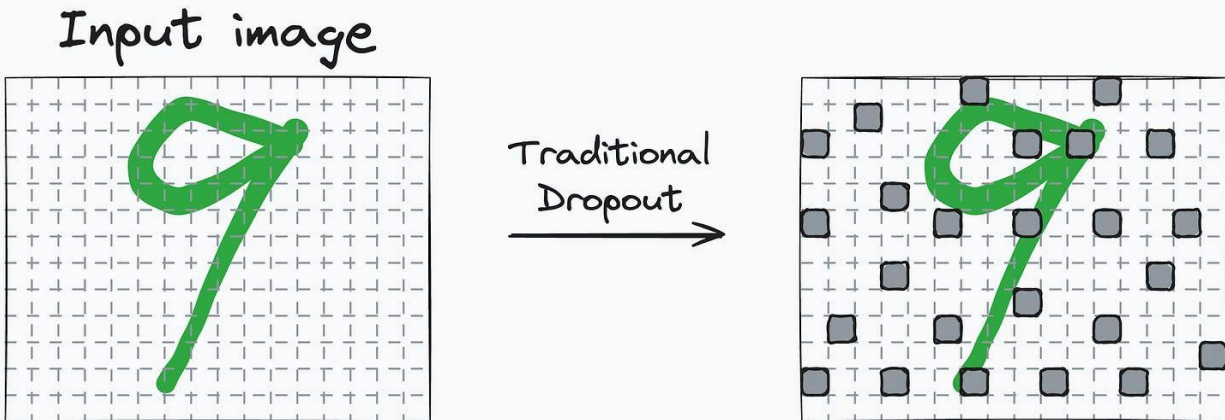
The problem of using Dropout in CNNs

The core operation that makes CNNs so powerful is convolution, which allows them to capture local patterns, such as edges and textures, and helps extract relevant information from the input.

From a purely mathematical perspective, we slide a filter (shown in yellow below) over the input (shown in green below) and take the element-wise sum between the filter and the overlapped input to get the convolution output:



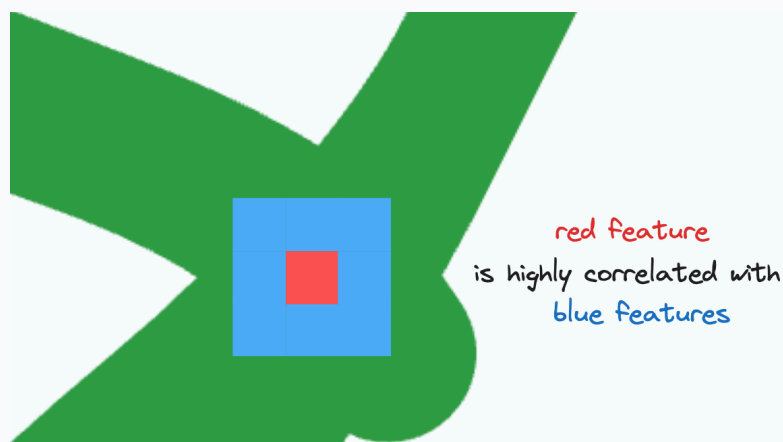
Here, if we were to apply the traditional Dropout, the input features would look something like this:



In fully connected layers, we zero out neurons. In CNNs, however, we randomly zero out the pixel values before convolution, as depicted above.

But this isn't found to be that effective specifically for convolution layers. To understand this, consider we have some image data. In every image, we would find that nearby features (or pixels) are highly correlated spatially.

For instance, imagine zooming in on the pixel level of the digit '9'. Here, we would notice that the red pixel (or feature) is highly correlated with other features in its vicinity:

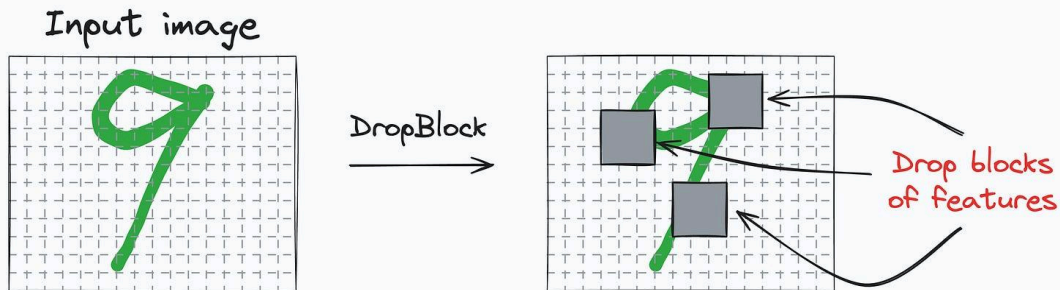


Thus, dropping the red feature using Dropout will likely have no effect and its information can still be sent to the next layer.

Simply put, the nature of the convolution operation defeats the entire purpose of the traditional Dropout procedure.

The solution

DropBlock is a much better, effective, and intuitive way to regularize CNNs. The core idea in DropBlock is to drop a contiguous region of features (or pixels) rather than individual pixels. This is depicted below:

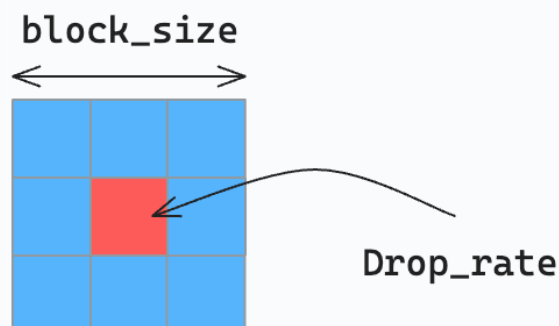


Similar to Dropout in fully connected layers, wherein the network tries to generate more robust ways to fit the data in the absence of some activations, in the case of DropBlock, the convolution layers get more robust to fit the data despite the absence of a block of features.

Moreover, the idea of DropBlock also makes intuitive sense — if a contiguous region of a feature is dropped, the problem of using Dropout with convolution operation can be avoided.

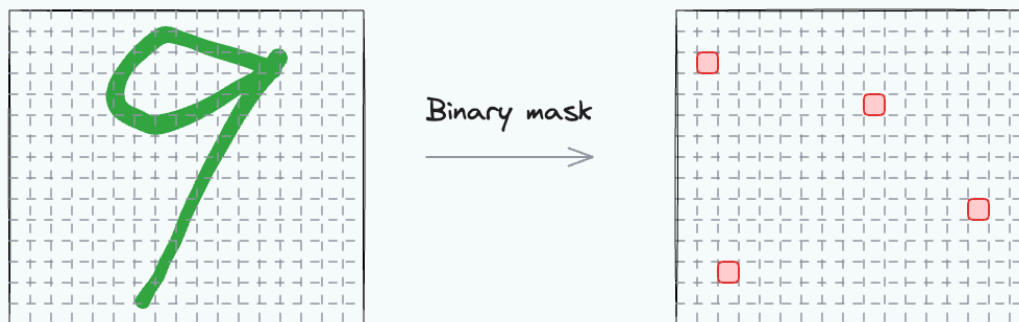
DropBlock parameters

DropBlock has two main parameters:

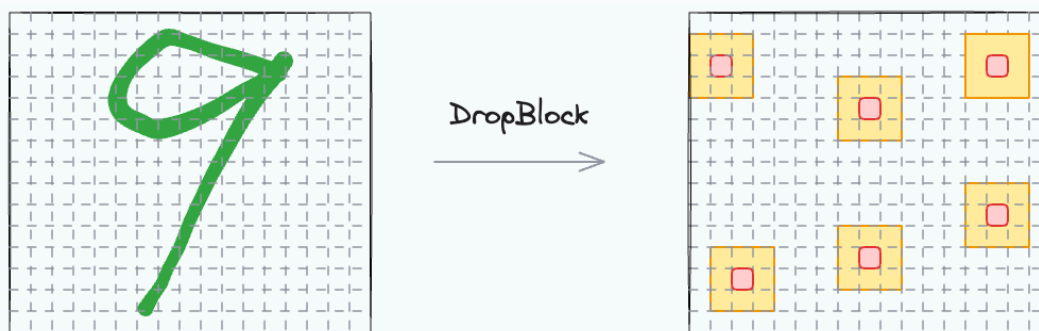


- **Block_size:** The size of the box to be dropped.
- **Drop_rate:** The drop probability of the central pixel.

To apply DropBlock, first, we create a binary mask on the input sampled from the Bernoulli distribution:



Next, we create a block of size `block_size*block_size` which has the sampled pixels at the center:



The efficacy of DropBlock over Dropout is evident from the results table below:

Model	top-1(%)	top-5(%)
ResNet-50	76.51 ± 0.07	93.20 ± 0.05
ResNet-50 + dropout (kp=0.7) [1]	76.80 ± 0.04	93.41 ± 0.04
ResNet-50 + DropPath (kp=0.9) [17]	77.10 ± 0.08	93.50 ± 0.05
ResNet-50 + SpatialDropout (kp=0.9) [20]	77.41 ± 0.04	93.74 ± 0.02
ResNet-50 + Cutout [23]	76.52 ± 0.07	93.21 ± 0.04
ResNet-50 + AutoAugment [27]	77.63	93.82
ResNet-50 + label smoothing (0.1) [28]	77.17 ± 0.05	93.45 ± 0.03
ResNet-50 + DropBlock, (kp=0.9)	78.13 ± 0.05	94.02 ± 0.02
ResNet-50 + DropBlock (kp=0.9) + label smoothing (0.1)	78.35 ± 0.05	94.15 ± 0.03

1.33% gain (from ResNet-50 + dropout to ResNet-50 + DropBlock)

1.55% gain (from ResNet-50 + dropout to ResNet-50 + DropBlock + label smoothing)

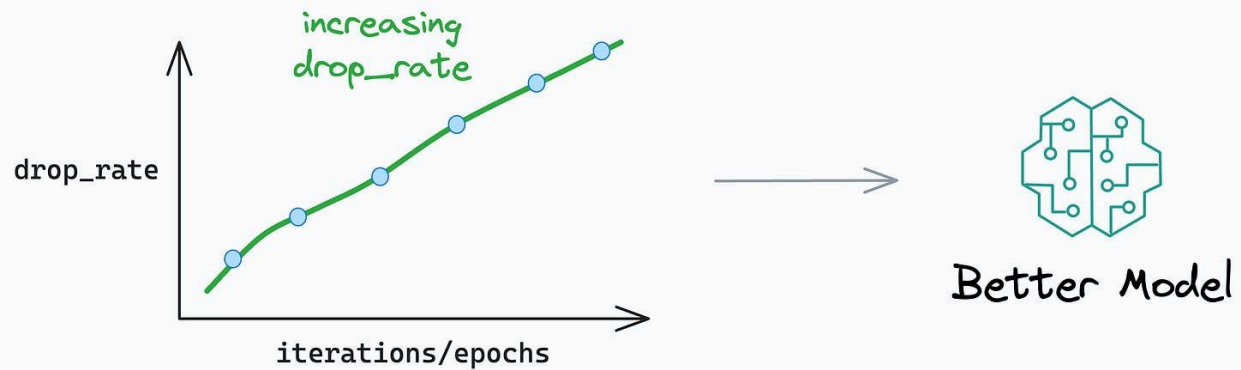
On the ImageNet classification dataset:

- DropBlock provides a 1.33% gain over Dropout.
- DropBlock with Label smoothing (discussed in the last chapter) provides a 1.55% gain over Dropout.

Thankfully, DropBlock is also integrated with PyTorch.

There's also a library for DropBlock, called “`dropblock`,” which also provides the linear scheduler for `drop_rate`.

So the thing is that the researchers who proposed DropBlock found the technique to be more effective when the `drop_rate` was increased gradually.



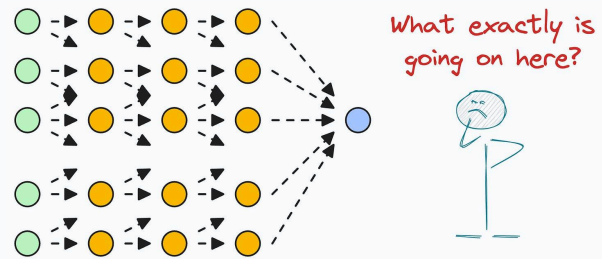
The DropBlock library implements the scheduler. But of course, there are ways to do this in PyTorch as well. So it's entirely up to you which implementation you want to use:

- DropBlock PyTorch: <https://bit.ly/3xZfT3E>.
- DropBlock library: <https://github.com/miguelvr/dropblock>.

What Hidden Layers and Activation Functions Actually Do

Everyone knows the objective of an activation function in a neural network. They let the network learn non-linear patterns. There is nothing new here, and I am sure you are aware of that too.

However, one thing I have often realized is that most people struggle to build an intuitive understanding of what exactly a neural network consistently tries to achieve during its layer-after-layer transformations.

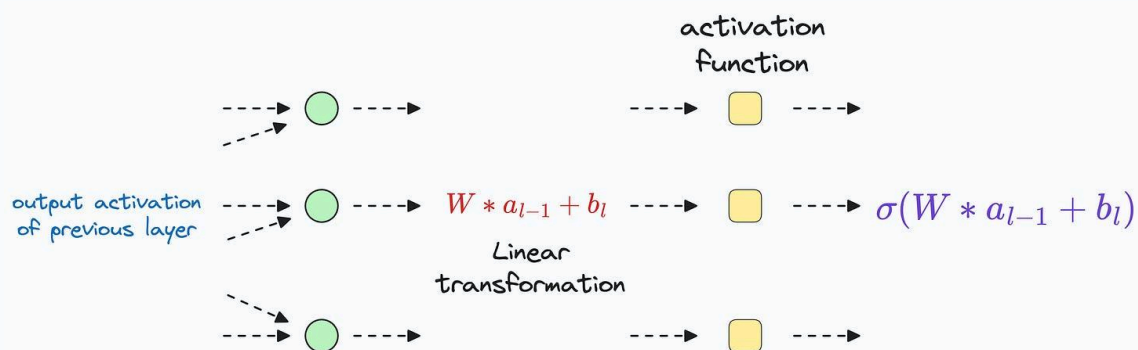


In this chapter, let me share a unique perspective on this, which would really help you understand the internal workings of a neural network.

I have supported this chapter with plenty of visuals for better understanding. Also, for simplicity, we shall consider a binary classification use case.

Background

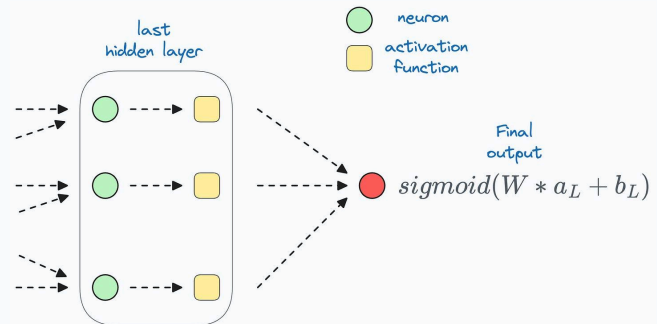
The data undergoes a series of transformations at each hidden layer:



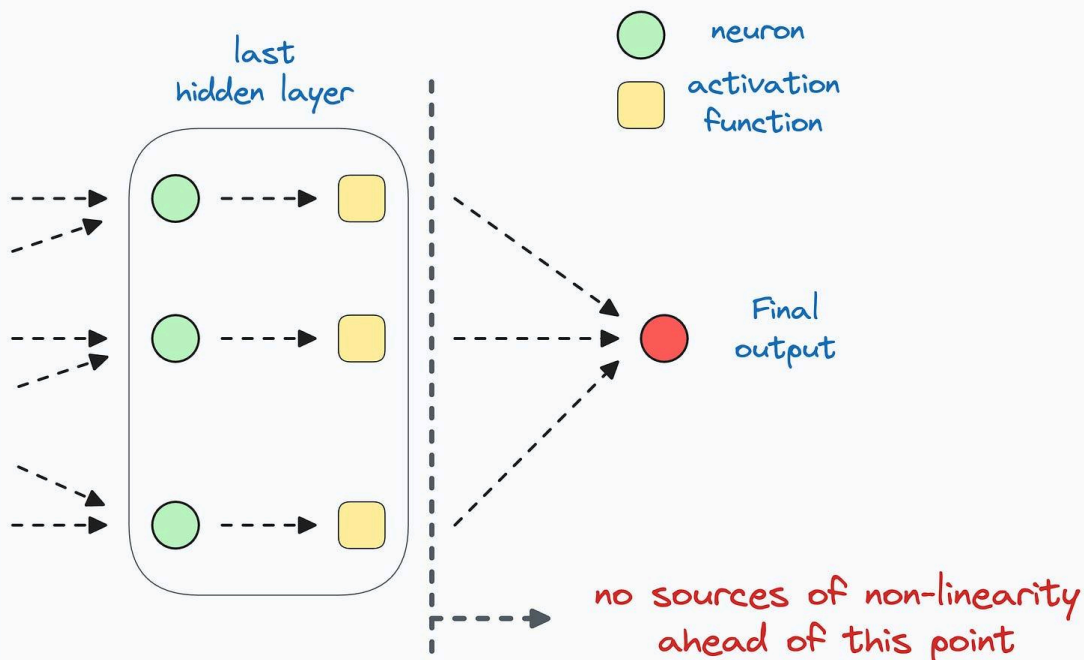
- Linear transformation of the data obtained from the previous layer
- ...followed by a non-linearity using an activation function — ReLU, Sigmoid, Tanh, etc.

The above transformations are performed on every hidden layer of a neural network. Now, notice something here.

Assume that we just applied the above data transformation on the **very last hidden layer** of the neural network. Once we do that, the activations progress toward the output layer of the network for one final transformation, which is entirely linear.



The above transformation is entirely linear because all sources of non-linearity (activation functions) exist on or before the last hidden layer. And during the forward pass, once the data leaves the last hidden layer, there is no further scope for non-linearity.



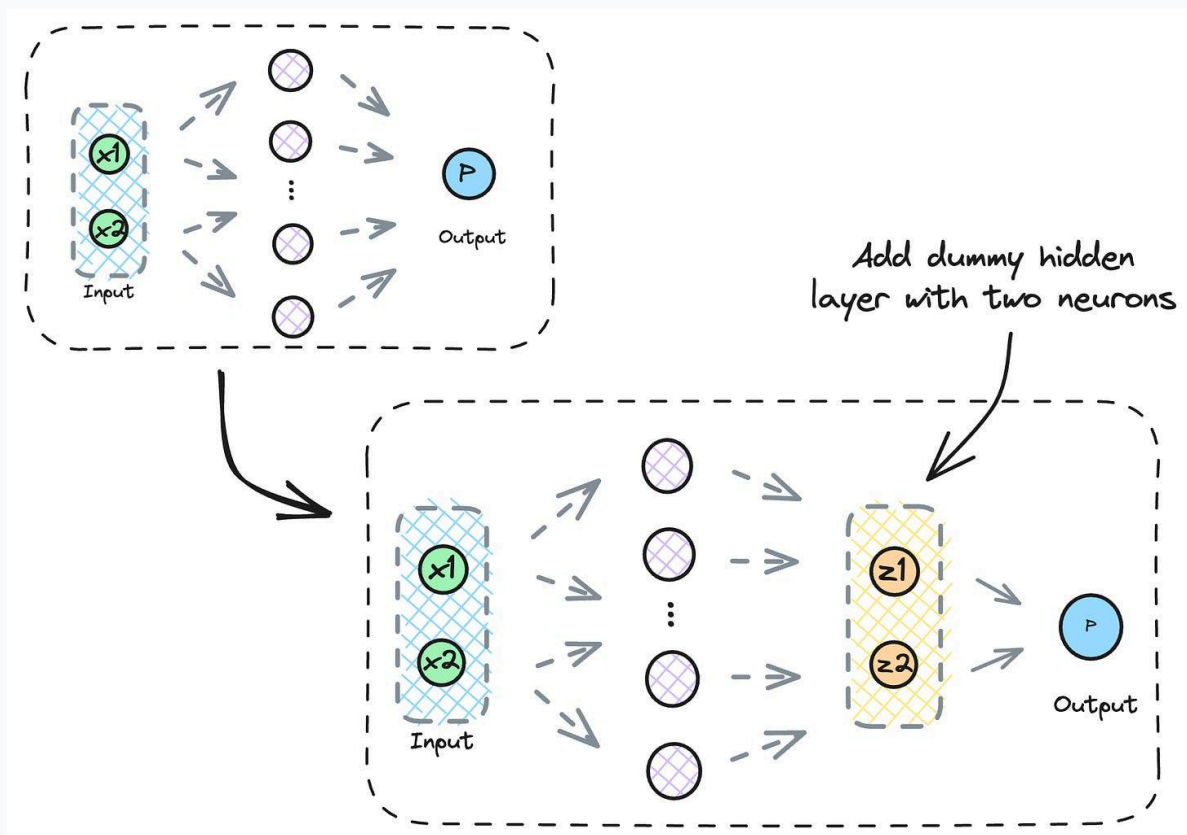
Thus, to make accurate predictions, the data received by the output layer from the last hidden layer **MUST BE** linearly separable.

To summarize....

While transforming the data through all its hidden layers and just before reaching the output layer, a neural network is constantly hustling to project the data to a space where it somehow becomes linearly separable. If it does, the output layer becomes analogous to a logistic regression model, which can easily handle this linearly separable data.

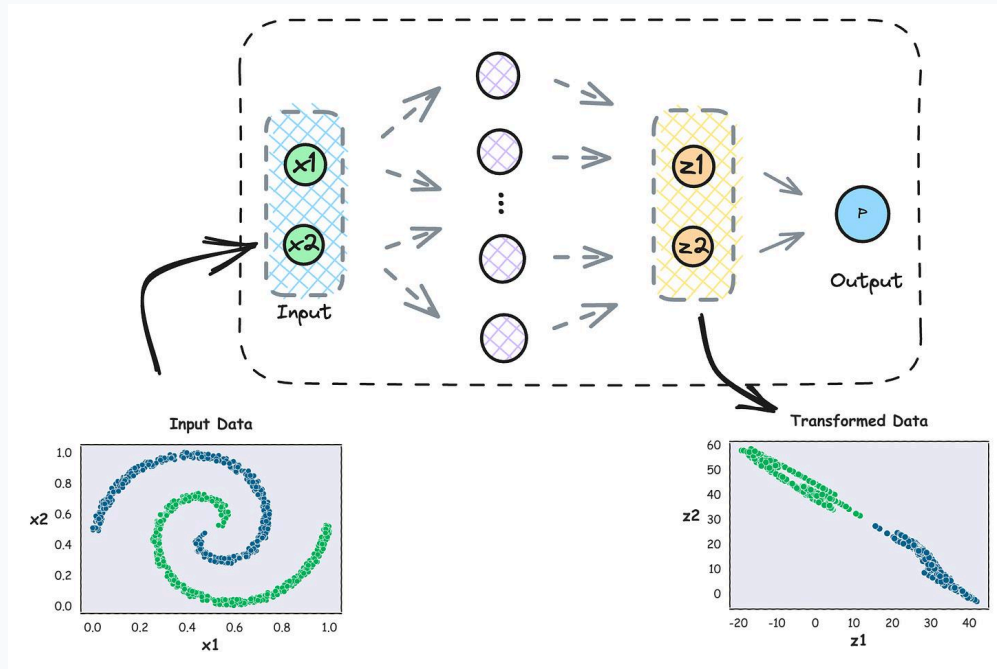
In fact, we can also verify this experimentally.

To visualize the input transformation, we can add a dummy hidden layer with just two neurons right before the output layer and train the neural network again.



Why do we add a layer with just two neurons?

This way, we can easily visualize the transformation. We expect that if we plot the activations of this 2D dummy hidden layer, they must be linearly separable. The below visual precisely depicts this.



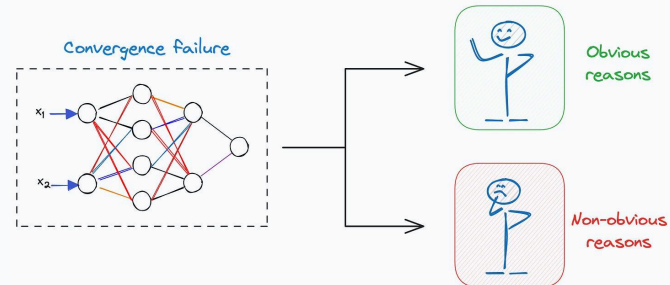
As we notice above, while the input data was linearly inseparable, the input received by the output layer is indeed linearly separable.

This transformed data can be easily handled by the output classification layer.

And this shows that all a neural network is trying to do is transform the data into a linearly separable form before reaching the output layer.

Shuffle Data Before Training

Deep learning models may fail to converge due to various reasons. Some causes are obvious and common, and therefore, quickly rectifiable, like too high/low learning rate, no data normalization, no batch normalization, etc.

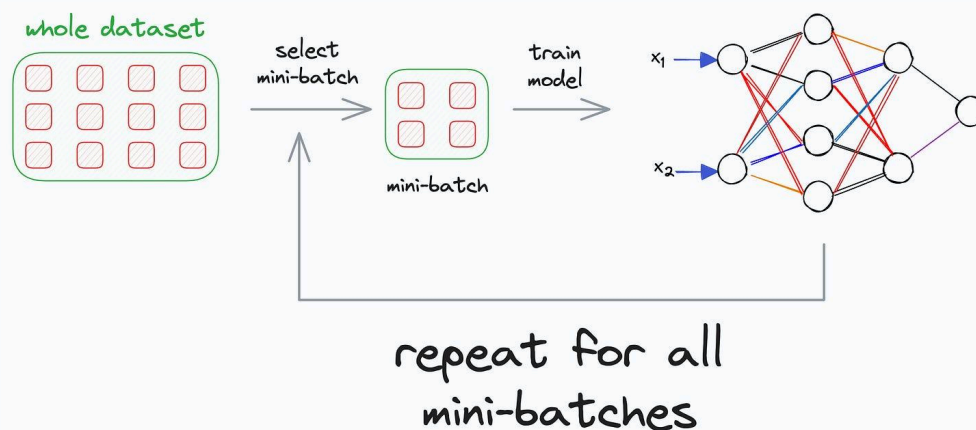


But the problem arises when the cause isn't that apparent. Therefore, it may take some serious time to debug if you are unaware of them. In this chapter, I want to talk about one such data-related mistake, which I once committed during my early days in machine learning. Admittedly, it took me quite some time to figure it out back then because I had no idea about the issue.

Experiment

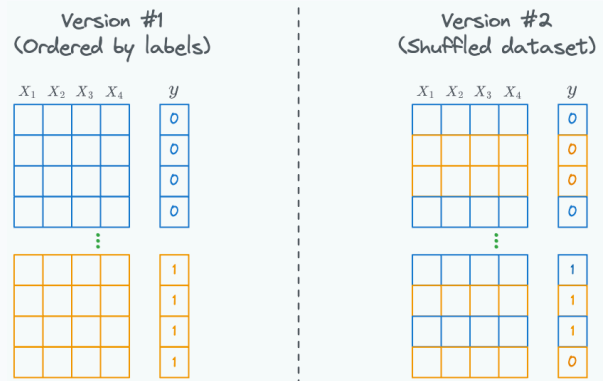
Consider a classification neural network trained using mini-batch gradient descent.

Mini-batch gradient descent: Update network weights using a few data points at a time.



Here, we train two different neural networks:

- Version 1: The dataset is ordered by labels.
- Version 2: The dataset is properly shuffled by labels.



And, of course, before training, we ensure that both networks had the same initial weights, learning rate, and other settings.

The image depicts the epoch-by-epoch performance of the two models. On the left, we have the model trained on label-ordered data, and the one on the right was trained on the shuffled dataset.

# Data ordered by labels			# Data shuffled		
Epoch	Loss	Accuracy	Epoch	Loss	Accuracy
[10/100]	109.48	50.00%	[10/100]	94.36	58.70%
[20/100]	108.35	50.00%	[20/100]	30.56	99.68%
[30/100]	106.48	61.33%	[30/100]	0.97	100.00%
[40/100]	105.95	61.00%	[40/100]	0.21	100.00%
[50/100]	103.84	61.78%	[50/100]	0.06	100.00%
[60/100]	110.06	61.88%	[60/100]	0.02	100.00%
[70/100]	112.30	62.14%	[70/100]	0.01	100.00%
[80/100]	102.10	62.40%	[80/100]	0.00	100.00%
[90/100]	101.61	62.88%	[90/100]	0.00	100.00%
[100/100]	116.14	61.91%	[100/100]	0.00	100.00%

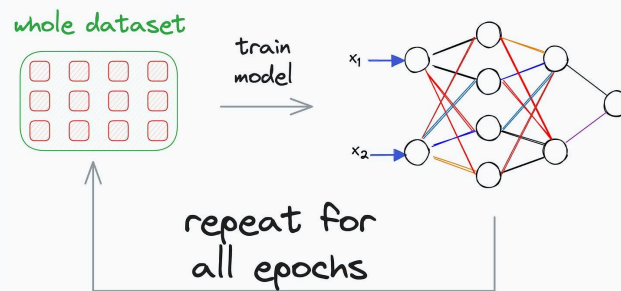
It is clear that the model receiving a label-ordered dataset miserably fails to converge while the other model, although overfits, shows that model has been learn effectively.

Why does that happen?

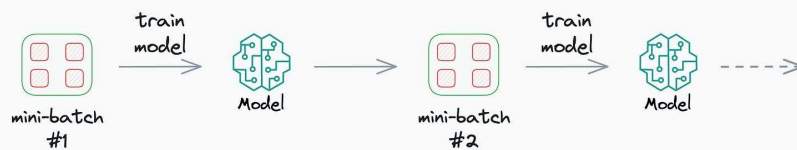
Now, if you think about it for a second, overall, both models received the same data, didn't they? Yet, the order in which the data was fed to these models totally determined their performance. I vividly remember that when I faced this issue, I knew that my data was ordered by labels.

Yet, it never occurred to me that ordering may influence the model performance because the data will always be the same regardless of the ordering.

But later, I realized that this point will only be valid when the model sees the entire data and updates the model weights in one go, i.e., in batch gradient descent, as depicted in this image.



But in the case of mini-batch gradient descent, the weights are updated after every mini-batch. Thus, the prediction and weight update on a subsequent mini-batch is influenced by the previous mini-batches.



In the context of label-ordered data, where samples of the same class are grouped together, mini-batch gradient descent will lead the model to learn patterns specific to the class it excessively saw early on in training. In contrast, randomly ordered data ensures that each mini-batch contains a balanced representation of classes. This allows the model to learn a more comprehensive set of features throughout the training process.

Of course, the idea of shuffling is not valid for time-series datasets as their temporal structure is important. The good thing is that if you happen to use, say, PyTorch DataLoader, you are safe. This is because it already implements shuffling. But if you have a custom implementation, ensure that you are not making any such error.

Before I end, one thing that you must ALWAYS remember when training neural networks is that these models can proficiently learn entirely non-existing patterns about your dataset. So never give them any chance to do so.

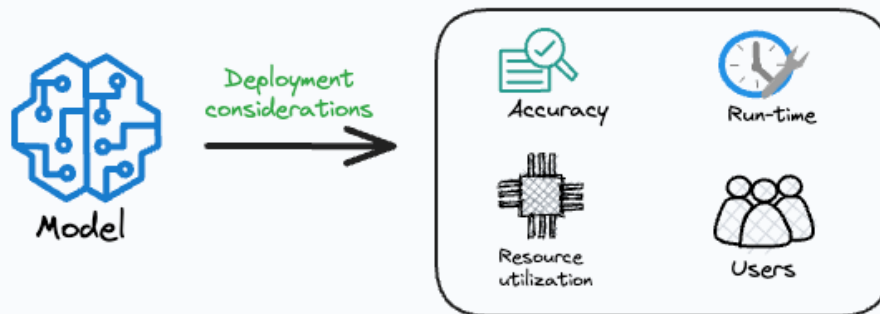
Model compression

Knowledge Distillation for Model Compression

Model accuracy alone (or an equivalent performance metric) rarely determines which model will be deployed.

This is because we also consider several operational metrics, such as:

- Inference Latency: Time taken by the model to return a prediction.
- Model size: The memory occupied by the model.
- Ease of scalability, etc.



In this chapter, let me share a technique (with a demo) called knowledge distillation, which is commonly used to compress ML models and contribute to the above operational metrics.

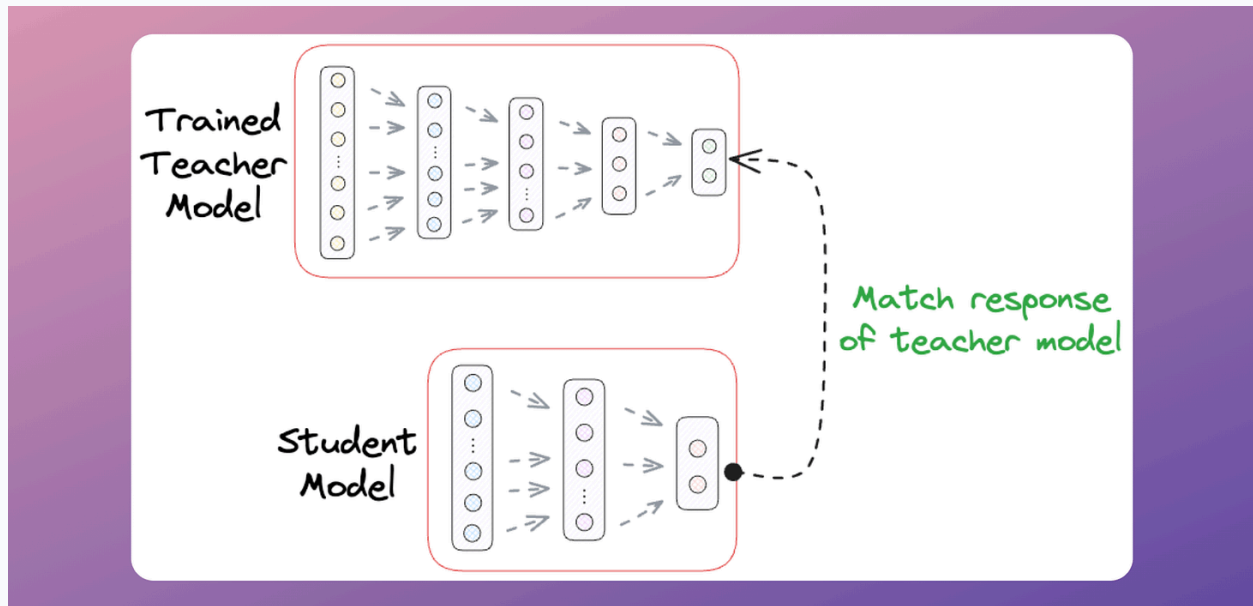
What is knowledge distillation?

In a gist, the idea is to train a smaller/simpler model (called the “student” model) that mimics the behavior of a larger/complex model (called the “teacher” model).



This involves two steps:

- Train the teacher model as we typically would.
- Train a student model that matches the output of the teacher model.



If we compare it to an academic teacher-student scenario, the student may not be as performant as the teacher.

But with consistent training, a smaller model may get (almost) as good as the larger one.

A classic example of a model developed in this way is DistillBERT. It is a student model of BERT.

- DistilBERT is approximately 40% smaller than BERT, which is a massive difference in size.
- Still, it retains approximately 97% of the BERT's capabilities.

Next, let's look at a demo.